High-dimensional Regret Minimization

Junyu Liao
The Hong Kong University of Science and Technology
jliaoak@connect.ust.hk

Ashwin Lall
Denison University
lalla@denison.edu

Mitsunori Ogihara
University of Miami
m.ogihara@miami.edu

Raymond Chi-Wing Wong
The Hong Kong University of Science and Technology
raywong@cse.ust.hk

arXiv:2512.24078v2 [cs.DB] 9 Nov 2026

***Abstract*—Multi-criteria decision making over large databases is fundamental to many real-world applications. Interactive regret minimization has emerged recently as a promising approach that combines the controllable output size of top-k queries with the flexibility of skyline queries, without requiring users to explicitly specify a utility function. Instead, the system iteratively presents a small set of options, learns the user's implicit preferences from their selections, and progressively refines its recommendations.**

However, modern applications such as housing search and financial product selection often involve hundreds of attributes. Existing interactive methods either fail to scale to such high-dimensional data or require excessive user interaction, frequently exceeding 1000 rounds. We address this limitation with FHDR (Fast High-Dimensional Reduction), a framework that exploits sparse linear preferences to discover key attributes through lightweight interactions. We also propose AttributeSubset, an efficient single-round algorithm that returns a compact recommendation set without further feedback. In our experiments on real datasets, FHDR completes in under 0.03s and requires fewer than 30 interaction rounds under the default setting.

Extensive experiments on both synthetic and real-world datasets show that FHDR outperforms the best-known baselines by at least one order of magnitude in execution time and by up to several orders of magnitude in interaction cost. These results establish FHDR as a new state of the art for scalable interactive regret minimization.

***Index Terms*—regret minimization, user interaction, multi-criteria decision making**

I. Introduction

Multi-criteria decision making over large databases is important in many real-world applications. A common approach to facilitating user decision-making in databases is to model user preference as a *utility function* [1]–[3], which assigns each tuple a numerical score. Knowledge of this function—either explicit or implicit—enables systems to reduce user effort in identifying preferred tuples. Two major paradigms built on this assumption are (1) the *top-k query* [4]–[8] and (2) the *skyline query* [9]–[13]. (1) In a top-k query, the user explicitly provides a utility function and an integer k, and the system returns the k highest-scoring tuples. While conceptually straightforward, this approach is impractical when the user cannot specify precise weights in the utility function—especially in high-dimensional data, where hundreds of attributes make this specification infeasible. (2) The skyline query, by contrast, assumes only that the utility function is component-wise monotonic: if one tuple is no worse than another in every attribute, its utility is at least as high. The system thus returns all skyline points, i.e., tuples not dominated by any other. Although this avoids explicit utility specification, the output can become unmanageably large. As dimensionality increases, domination becomes less common and the skyline typically contains more tuples, potentially overwhelming the user.

Recently, the k-regret query [3], [14]–[17] has emerged as a promising solution to the limitations of top-k and skyline queries. The method selects k representative tuples such that, for any possible user preference, the best tuple in the returned set closely approximates the global optimum in the database. The regret ratio measures the relative utility loss incurred by choosing from the returned set rather than the entire database (Section III). The objective is to minimize the maximum regret ratio over the admissible utility functions.

These frameworks operate in a single round, presenting their results at once. This naturally raises the question: can multiple rounds of interaction further reduce regret or output size? To address this, interactive regret minimization frameworks [2], [18] have been proposed, where the system iteratively presents a few candidate tuples per round, learns the user's implicit utility function from their choices, and progressively refines subsequent recommendations.

Most previous studies have focused on databases with a small number of attributes, typically 2-10, and often require users to specify their attributes of interest beforehand. In contrast, this paper considers a *high-dimensional setting* with more than a hundred attributes and no prior attribute selection. These scenarios are increasingly common nowadays, e.g., in housing markets (price, size, location, etc.) and financial products (interest rate, liquidity, historical performance, etc.). In these domains, the high dimensionality poses significant challenges: computational costs grow exponentially, theoretical guarantees become weaker, and traditional algorithms often fail to provide meaningful representative subsets. There is an urgent need for efficient and effective algorithms that address these challenges.

We want algorithms that satisfy the following requirements:

- *Quality.* Return a small, manageable set with low regret,

ideally identifying the user's favorite tuple in the database.

- *Efficiency.* Execute within a reasonable time on high-dimensional datasets that may contain millions of tuples.
- *Usability.* Require few interactions, using simple, intuitive questions that remain manageable as dimensionality grows.

Our work is motivated by the observation that users typically base decisions on a small subset of salient attributes rather than the full feature set. For example, a house may be described by hundreds of attributes, including floor area, nearby amenities, and energy efficiency, yet buyers often focus primarily on a few factors such as price, size, and location. This behavior is well documented in cognitive psychology [19]–[22] and conjoint analysis [23]–[26], and is further supported by our survey.

In psychology, the "Take-The-Best" heuristic [19], [20] shows that people make accurate decisions using only a few salient attributes. It ranks attributes by importance and bases each choice on the first attributes that distinguish the alternatives, ignoring the rest. Despite its simplicity, it can outperform more complex models, including multiple regression and weighted integration [21], [22]. This finding supports our high-dimensional k-regret setting and motivates the assumption that the true utility vector is sparse, with most weight concentrated on a small subset of attributes.

Conjoint analysis similarly estimates preferences over multiple attributes from evaluations or choices among alternatives [23]–[26]. Estimated attribute contributions yield relative importance scores, providing evidence that importance can be concentrated on a small subset of attributes. In Schaupp and Bélanger's study, the five highest-ranked attributes account for 84.2% of the reported relative importance [24]. This concentration of importance motivates our sparse-utility model.

To examine how many attributes users consider in decision making, we surveyed 30 participants using the Car, House, and NBA datasets described in Section VI. Participants were shown a set of options and asked to identify the attributes they considered when comparing them. They could select any number of attributes, with no minimum or maximum imposed. The datasets contain 57, 33, and 104 attributes, respectively. Figure 1 summarizes the number of attributes selected for each dataset and shows that most participants relied on only a small subset. For the Car dataset, 12 participants (40%) considered exactly 3 of the 57 attributes, with *Real-World MPG*, *Model Year*, and *Engine Displacement* being the three most frequently selected. Across all datasets, more than 80% of participants considered at most 5 attributes. These results provide empirical support for our assumption that users' effective utility functions are sparse.

Motivated by this, we introduce FHDR (Fast High-Dimensional Reduction), a generic framework for k-regret minimization in high-dimensional settings. Our major contributions are as follows:

- We present FHDR, a framework for high-dimensional regret minimization under sparse linear preferences. It handles datasets of arbitrary original dimensionality and identifies the user's favorite tuple with sufficient feedback, requiring substantially fewer questions in the evaluated comparisons.
- We implicitly discover the user's key attributes through tuple comparisons, avoiding upfront attribute selection requirement in previous papers.
- We support flexible interaction, including zero feedback. The user can skip or stop answering questions at any time, ensuring a user-friendly interaction process.
- We propose a single-round algorithm ATTRIBUTESUBSET for any dimensionality, which is more than 10 times faster than existing single-round algorithms under high-dimensional settings.
- We conduct extensive experiments on synthetic and real datasets to evaluate regret ratio, computation time, and interaction cost. A user study on Car further evaluates FHDR's end-to-end recommendation quality and usability.

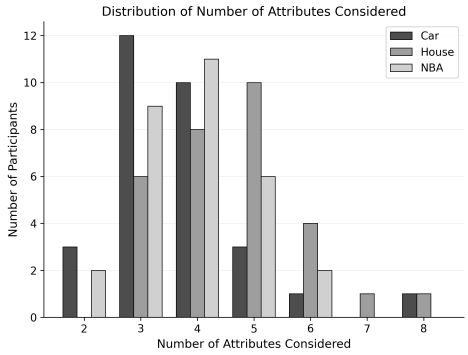


Fig. 1: Survey on Number of Attributes Considered

The rest of the paper is organized as follows: Section II discusses related work, and Section III gives the problem definition. The complete algorithm for high-dimensional regret minimization is presented in Section IV, while the solution for limited user feedback is given in Section V. Section VI shows the experiment results and Section VII concludes the paper.

II. Related Work

The k-regret query [3], also known as the regret-minimizing set query, addresses the limitations of top-k and skyline queries. It returns k representative tuples such that, for any unknown nonnegative linear utility function over d attributes, the best returned tuple closely approximates the global optimum. Its objective is to minimize the maximum regret ratio defined in Section III, thereby avoiding explicit utility specification while controlling the output size. However, finding an optimal k-regret set is NP-hard [14], [27], [28], particularly in high dimensions. Early methods, including CUBE [3], GREEDY [3], and GEOGREEDY [16], either lack theoretical guarantees or perform poorly in practice. Later approaches such as HD-GREEDY [15] and ε-KERNEL [27] improve efficiency but offer weak bounds: HD-GREEDY provides only a loose upper bound, while the bound of ε-KERNEL may exceed 1 even for $d < 10$, making it uninformative in practice.

Recent advances have extended regret minimization to high-dimensional settings. Xie et al. [17] proposed the SPHERE algorithm, which achieves an asymptotically optimal upper

bound on the maximum regret ratio without restricting data dimensionality. Although earlier methods such as Cube [3] also provide theoretical guarantees that apply across dimensions, Sphere is distinguished by offering an asymptotically optimal bound while also demonstrating strong empirical performance, making it a state-of-the-art single-round solution. However, its performance degrades as dimensionality increases due to two limitations: (1) it requires the output size k to be at least the dimensionality d, which is impractical in high-dimensional applications, and (2) its runtime grows sub-exponentially with d, rendering it inefficient for large datasets. These issues motivate the development of complementary approaches for scalable high-dimensional decision making.

Another line of work studies interactive regret minimization, which replaces one-shot query processing with multi-round interaction to progressively infer the user's implicit utility function. The first such framework, UTILITYAPPROX [2], repeatedly presents a small set of tuples and asks the user to select the preferred one. Each response refines the estimated utility function, allowing subsequent queries to focus on more promising tuples and reduce regret for a fixed output size [2]. However, UTILITYAPPROX relies on synthetic tuples, which may frustrate users when a preferred option is not real, and requires an impractically large number of questions in high-dimensional settings.

Xie et al. [18] addressed this limitation with a strongly truthful framework that displays only real tuples, improving user trust and simplifying preference elicitation. Their algorithms, including UH-RANDOM and UH-SIMPLEX, converge to near-optimal results within a bounded number of rounds and achieve an asymptotically optimal question complexity in two dimensions. However, their scalability is limited in higher dimensions by numerical instability in convex-hull computations. To overcome this issue, these methods require users to explicitly specify a small subset of relevant attributes, which imposes substantial burden in high-dimensional settings, as will be justified in later paragraphs.

Several other interactive methods have also been proposed. ACTIVE-RANKING [29] adaptively selects pairwise comparisons to recover a complete ranking under a Euclidean embedding assumption. However, it targets full-ranking recovery rather than regret minimization and assumes that the data are in general position, which may not hold in practical databases. PREFERENCE-LEARNING [30] similarly reconstructs the entire utility function through adaptive comparisons, requiring queries that may be irrelevant to identifying the user's preferred tuples. More recently, Wang et al. [31] proposed HD-PI and RH for identifying top-k tuples. Although these methods provide dimension-independent guarantees under ideal conditions, their scalability deteriorates as dimensionality increases because convex-hull and skyline frontiers grow rapidly, increasing both computational and interaction costs. Chen and Wong [32] further study best-tuple retrieval under random interaction errors through VERIFY-POINT and VERIFY-SPACE. These methods provide probabilistic retrieval guarantees over a predefined attribute space, whereas FHDR discovers key attributes implicitly from high-dimensional data.

A further limitation of existing works is upfront attribute selection. For example, a user may be asked to select 5 attributes from 100 directly, before seeing actual candidate tuples. Do et al. [33] simplify interaction by comparing two attributes at a time, but still require users to select their attributes of interest. Prior work often treats this step as trivial preprocessing, while our survey and user study indicate that upfront attribute selection can be burdensome.

This selection can impose a substantial cognitive burden: users must inspect and compare many attributes while working memory is limited [34]. Moreover, users' preferences are often constructed during actual decision-making rather than specified in advance [35], so selection without concrete candidate tuples risks prematurely excluding relevant attributes. On the other hand, partial-profile designs, which show only a subset of attributes at a time, have been reported to reduce decision complexity and choice randomness while improving response efficiency [36]. Comparisons between concrete tuples also offer a more intuitive way to elicit preferences [2], [18], [31], [32]. For example, a user may be unable to state abstractly whether "location" is globally more important than "floor area", yet can readily choose between two houses once the trade-off is instantiated. In this way, comparison-based interaction reveals preferences implicitly with less cognitive effort. These observations motivate FHDR's implicit discovery of key attributes through tuple comparisons.

Compared with existing works, FHDR natively supports high-dimensional regret minimization and maintains strong performance as dimensionality increases, addressing key scalability barriers. Its user-friendly design requires no upfront attribute selection and allows users to skip or stop answering questions at any time. Table I summarizes the advantages of our method, alongside comparison with existing methods. By combining high-dimensional scalability, computational efficiency, and flexible interaction, FHDR advances the state of the art and enables practical and accurate decision support for complex, real-world multi-criteria applications.

III. PROBLEM DEFINITION

The input is a preprocessed dataset $X \subseteq [0,1]^d$ of n points. Here, d denotes the number of attributes. We assume that a user is only interested in d_{int} among all d attributes, where d_{int} is a small number compared to d (e.g., $d_{\text{int}} = 3$ and $d = 100$).

A. Terminology

We refer to a database object as "*tuple*" and "*point*" interchangeably throughout the paper. We denote the i-th dimensional value of a point p by $p[i]$ where $i \in [1, d]$. For an attribute set $\mathcal{D}$, let $p_{\mathcal{D}}$ denote the projection of p onto $\mathcal{D}$, and let $X_{\mathcal{D}} = \{p_{\mathcal{D}} : p \in X\}$. We remove constant attributes and apply *min-max* normalization to each remaining attribute. Let a_i and b_i denote the minimum and maximum raw values of attribute i. A raw value $x[i]$ is mapped to $p[i] = (x[i] - a_i)/(b_i - a_i)$ when larger values are preferred, and to $p[i] = (b_i - x[i])/(b_i - a_i)$ when smaller values are

Method	Practical high-dimensional support	Exact-favorite guarantee	Implicit key-attribute discovery	User-controlled output size	Output at any interaction budget
SPHERE [17]	✓	×	×	×	✓
UTILITYAPPROX [2]	✓	✓	×	×	×
UH-RANDOM [18]	×	✓	×	×	×
HD-PI / RH [31]	×†	✓	×	✓	×
VERIFY-POINT [32]	×†	✓	×	✓	×
BREAKPOINT [33]	✓	×	×	×	✓
FHDR	✓	✓	✓	✓	✓

TABLE I: Capabilities of representative methods. Practical high-dimensional support reflects practical computation and interaction costs in the evaluated high-dimensional settings; ✓ denotes partial support with substantial practical limitations. † Execution entries concern the tested representatives HD-PI and VERIFY-POINT, respectively. Detailed qualifications and reproduction results appear in Appendix C.

House	D_1	D_2	D_3	D_4	D_5	Utility $f(p)$
p_1	0.84	0.61	0.93	0.70	0.31	0.782
p_2	0.59	0.95	0.77	0.86	0.79	0.761
p_3	0.69	0.84	1.00	0.99	0.55	0.820
p_4	1.00	0.64	0.68	0.45	1.00	0.794
p_5	0.74	1.00	0.44	1.00	0.73	0.756

TABLE II: House dataset with five attributes and utility values for $u = (0.40, 0.35, 0.25, 0, 0)$ ($d_{\text{int}} = 3$). The attributes $D_1 \sim D_5$ denote *price*, *size*, *commute time*, *age* and *condition score*, respectively.

preferred, as with *price*. Thus, every retained attribute ranges from 0 to 1, attains both endpoints, and has larger normalized values preferred. Utilities and regret ratios are defined on this normalized representation. As an example, Table II illustrates a database $X = \{p_1, p_2, p_3, p_4, p_5\}$ of 5 points ($n = 5$) where each point is described by 5 attributes ($d = 5$).

Following [1]–[3], [8], [16], we model user preferences by an unknown nonnegative linear utility function $f(p) = u \cdot p$, where $u \in \mathbb{R}^d_{\geq 0}$ and $u[i]$ represents the importance of attribute i. We use f and its utility vector u interchangeably.

Building on prior studies that individuals consider only a limited number of attributes in high-dimensional decision making tasks [19], [20], [23]–[26], along with our own user survey, we assume that u has exactly d_{int} positive entries, where $d_{\text{int}} \ll d$. From findings from previous studies and a user survey, we adopt an upper bound $d_{\max}$ of 5 (i.e., $d_{\text{int}} \leq d_{\max} = 5$). An attribute (dimension) is said to be *key* if the coefficient of that attribute in the utility vector $u[i] > 0$; and *non-key* otherwise.

The candidate attribute set $\mathcal{D}_{\text{cand}}$ contains all attributes not yet identified as non-key. Here, 'candidate' refers to attributes rather than tuples, to avoid confusion.

We define the *regret ratio* [3] as follows:

Definition III.1. [3] Given a nonempty set $S \subseteq X$ and a utility vector u, the *regret ratio* of S over X w.r.t. u, denoted by $\text{rr}_X(S, u)$, is defined as

$$\frac{\max_{p \in X} u \cdot p - \max_{p \in S} u \cdot p}{\max_{p \in X} u \cdot p} = 1 - \frac{\max_{p \in S} u \cdot p}{\max_{p \in X} u \cdot p}.$$

The regret ratio is invariant under positive scaling of u. We therefore normalize u so that $\sum_{i=1}^{d} u[i] = 1$ ($\|u\|_1 = 1$).

For any nonempty subset $S \subseteq X$, the regret ratio lies between 0% and 100%. Since each retained attribute attains 1 and $\|u\|_1 = 1$, we have $\max_{p \in X} u \cdot p \geq \max_i u[i] > 0$, so the regret ratio is well defined. A lower regret ratio means that the best returned tuple has utility closer to the dataset optimum. A tuple $p^* \in \arg\max_{p \in X} u \cdot p$ is a *favorite tuple*, also called an optimal or maximum-utility tuple.

Consider the example given in Table II. We have the utility vector $u = (0.40, 0.35, 0.25, 0, 0)$. Take the first point p_1 for illustration. Its corresponding utility w.r.t. u can be calculated as: $f(p_1) = u \cdot p_1 = 0.782$. The utilities of other points are also presented in Table II. The favorite tuple in X is p_3, with utility $f(p_3) = 0.820$. If $S = \{p_1, p_2\}$, then the regret ratio of S over X w.r.t. u is:

$$\text{rr}_X(S, u) = 1 - \frac{\max_{p \in S} u \cdot p}{\max_{p \in X} u \cdot p} = 1 - \frac{0.782}{0.820} \approx 4.63\%.$$

Notation is summarized in Appendix A.

B. Interaction

Following interactive regret-minimization frameworks [2], [18], [31], [32], FHDR learns preferences through tuple comparisons. It discovers key attributes and refines their relative utility weights without requiring explicit utility specification.

a) Format: Each question presents s tuples from X, denoted $\{p_1, p_2, \ldots, p_s\}$, described only by attributes in $\mathcal{D} \subseteq \{D_1, D_2, \ldots, D_d\}$, where $1 \leq |\mathcal{D}| \leq m$. Here, m is the maximum number of attributes displayed per question. The user either selects the most preferred tuple based solely on the displayed attributes or indicates that none of these attributes matters. We call the latter an *irrelevant-attributes response*. This response is distinct from stopping the interaction.

b) Strong truthfulness.: Following [18], an interactive algorithm is *strongly truthful* if it presents only existing objects to the user throughout the interaction. Our algorithm satisfies strong truthfulness because every tuple shown to the user is drawn from the original dataset.

c) Partial Utility Assumption: We assume a fixed utility vector u and user responses consistent with the partial-utility and attribute-relevance rules below. Users are instructed to treat omitted attributes as identical for each comparison [37], so their common contribution cancels. Following Do et al. [33], comparisons use the *partial utility* $f_{\mathcal{D}}(p) = \sum_{i:D_i \in \mathcal{D}} u[i]p[i]$, retaining the original coefficients of the displayed attributes. Users select a tuple maximizing partial utility when at least one displayed attribute is key; otherwise, they give an irrelevant-attributes response.

For the utility vector in Table II, displaying $\mathcal{D} = \{D_1, D_2, D_4\}$ gives $f_{\mathcal{D}}(p) = 0.40p[1] + 0.35p[2]$, since $u[4] = 0$. The five partial utilities are 0.550, 0.569, 0.570, 0.624, and 0.646, respectively, so the user selects p_5. If only D_4 and D_5 are displayed, both weights are zero and the user gives an irrelevant-attributes response.

d) Information Gained.: Each response conveys information about the relevance of attributes in $\mathcal{D}$:

- A tuple selection indicates that at least one displayed attribute is key.
- An irrelevant-attributes response indicates that $u[i] = 0$ for every displayed attribute $D_i \in \mathcal{D}$.

These responses refine $\mathcal{D}_{\text{cand}}$. A tuple selection also supplies comparisons on displayed coordinates that can constrain relative utility weights.

e) Early Termination.: Users may stop at any time. Before interactive Phase 3, including at zero feedback, AttributeSubset returns a recommendation set using the current candidate attributes. During interactive Phase 3, early stopping returns one recommendation based on the current feasible utility region; exact favorite-tuple retrieval requires sufficient feedback.

C. Objectives

We consider two objectives based on feedback.

a) Problem 1: : With sufficient feedback under the stated response model, our primary objective is to return a favorite tuple $p^* \in \arg\max_{p \in X} u \cdot p$.

b) Problem 2: : When interaction ends before interactive Phase 3, we seek a subset $S \subseteq X$ of user-specified size $|S| = K$ with a small regret ratio $\text{rr}_X(S, u)$, without further feedback [3]. This objective includes the zero-feedback case.

IV. Algorithm

We present FHDR for high-dimensional regret minimization under sparse linear preferences. The framework supports favorite-tuple retrieval with sufficient feedback (Problem 1 [III-C0a]) and a compact recommendation set under limited feedback (Problem 2 [III-C0b]). Section IV-A introduces the overall framework, Section IV-B describes attribute discovery, and Section IV-C presents the final output selection. Section IV-D summarizes the complete procedure. Detailed proofs are provided in Appendix B.

A. Overall Framework

Phases 1 and 2 exploit sparse preferences to discover relevant attributes without requiring upfront attribute selection. Phase 3 refines preferences over the retained attributes. Throughout discovery, we maintain a candidate set $\mathcal{D}_{\text{cand}}$, initialized with all attributes, and remove attributes identified as non-key.

a) Phase 1: Coarse Elimination.: Let $m \geq 1$ be an integer specifying the maximum number of attributes displayed per question. Starting with all d attributes, we partition them into blocks of at most m attributes. Each block is tested through a tuple comparison under the protocol in Section III. An irrelevant-attributes response eliminates the block; a tuple selection retains it for Phase 2.

b) Phase 2: Fine Selection.: We adapt generalized binary splitting [38] to identify individual key attributes among the remaining candidates. Tuple comparisons provide the relevance responses required by the group tests. With continued feedback, this phase ends when all candidates are classified or d_{max} key attributes have been identified.

c) Phase 3: Regret Minimization.: We adapt UH-Random [18] to refine preferences over the retained attributes. The feasible utility region incorporates comparisons on displayed coordinates from Phases 1 and 2 and is refined through further interaction. With sufficient feedback, the method identifies a favorite tuple.

At any time before Phase 3, if the user stops the interaction, AttributeSubset returns K tuples using the current candidate set $\mathcal{D}_{\text{cand}}$. At the extreme case with zero feedback, the candidate set could contain all d attributes. If the user stops during interactive Phase 3, we return one recommendation using the current feasible utility region.

B. Dimension Reduction

Dimension reduction seeks a small candidate set containing every key attribute, using as few questions as possible. Starting with all attributes, Phase 1 eliminates irrelevant blocks and Phase 2 identifies individual key attributes, while maintaining $|\mathcal{D}_{\text{cand}}| \geq d_{\text{int}}$.

*a) **User Interaction**:* In Phases 1 and 2, each question presents s tuples projected onto the attributes being tested. Only these attributes are displayed, so the number may vary between questions. Responses follow the partial-utility protocol in Section III and update the candidate set $\mathcal{D}_{\text{cand}}$.

*1) **Phase 1: Coarse Elimination**:* We partition the d attributes into $\lceil d/m \rceil$ disjoint blocks. For $i \in \{1, \ldots, \lceil d/m \rceil\}$, let $\mathcal{B}_i \subseteq \{D_1, \ldots, D_d\}$ denote the set of dimensions selected for the i-th Phase 1 question, defined by $\mathcal{B}_i = \{D_j : (i-1)m < j \leq \min(im, d)\}$. Each block contains at most m attributes. The final block is displayed without padding when it contains fewer than m attributes.

In each block, we:

1) Construct the projected dataset $X_{\mathcal{B}_i}$ by restricting each tuple in X to the dimensions in $\mathcal{B}_i$.
2) Randomly select and present s tuples from $X_{\mathcal{B}_i}$.

3) If the user selects a tuple, retain all attributes in $\mathcal{B}_i$ as potentially key. If the user gives an irrelevant-attributes response, remove the entire block from $\mathcal{D}_{\text{cand}}$.

Under the stated response model, every block containing a key attribute survives, while blocks containing only non-key attributes are eliminated. A complete Phase 1 uses at most $\lceil d/m \rceil$ questions and leaves $|\mathcal{D}_{\text{cand}}| \leq d_{\text{int}} m$: at most d_{int} blocks survive, each containing at most m attributes.

Example IV.1. For $d = 100$, $d_{\text{int}} = 3$, and $m = 7$, Phase 1 uses 15 questions and leaves at most 21 candidate attributes.

*2) **Phase 2: Fine Selection**:* Phase 2 applies generalized binary splitting [39], a group testing algorithm to the remaining candidate attributes, to identify the key attributes. Because the true number of key attributes is unknown, which is a required input parameter by the group testing algorithm, we initialize its upper bound as $d_{\text{left}} = d_{\max} = 5$ and pass it as the input. Each identified key attribute decreases d_{left} by one. Thus, d_{left} bounds the number of keys still to be found; for example, after identifying 2 keys, at most 3 remain.

Algorithm 1 details the procedure. If $|\mathcal{D}_{\text{cand}}| \leq 2d_{\text{left}} - 2$, we test attributes individually. Otherwise, we test a random group of size $g = \min\{2^\alpha, m\}$, where $\alpha = \lfloor \log_2((|\mathcal{D}_{\text{cand}}| - d_{\text{left}} + 1)/d_{\text{left}}) \rfloor$. An irrelevant-attributes response eliminates the group. A tuple selection triggers balanced binary splitting to isolate one key attribute, eliminating any attributes identified as non-key along the way. Each question displays only the tested attributes. If the user stops, all unclassified attributes and previously identified keys are retained.

Phase 2 ends when all candidates are classified, $d_{\max}$ key dimensions have been identified, or the user stops interacting. Under the default experimental settings, Phase 2 uses an average of 11 additional questions.

Example IV.2. We illustrate the first round of Phase 2 with $s = 2$ and $m = 6$. A response is *positive* if the user selects a tuple and *negative* if the user gives an irrelevant-attributes response. Each question displays only the tested attributes.

Suppose the dataset has 100 attributes, with true key attributes $\{D_1, D_7, D_{13}, D_{19}, D_{25}\}$ ($d_{\text{int}} = 5$), unknown to the algorithm. Starting from $\mathcal{D}_{\text{cand}} = \{D_1, \ldots, D_{100}\}$, Phase 1 with $m = 6$ retains the first five blocks, leaving $\mathcal{D}_{\text{cand}} = \{D_1, \ldots, D_{30}\}$. Initially, $d_{\text{left}} = d_{\max} = 5$. Since $30 > 2d_{\text{left}} - 2 = 8$, a group test is performed.

We compute $l = 30 - 5 + 1 = 26$, $\alpha = \lfloor \log_2(26/5) \rfloor = 2$, and $g = \min\{2^\alpha, m\} = 4$. Suppose the sampled set is $\mathcal{D} = \{D_1, D_2, D_3, D_4\}$. We present two tuples from $X_{\mathcal{D}}$ on these four attributes. Since $\mathcal{D}$ contains D_1, the response is *positive*, and binary splitting follows.

First, test $\{D_3, D_4\}$. The negative response identifies both attributes as non-key. The preceding positive group test therefore implies that $\{D_1, D_2\}$ contains a key attribute, without another question. Next, test $\{D_1\}$. The positive response identifies D_1 as key, while D_2 remains unclassified. Thus, the three questions display four, two, and one attributes, respectively.

Finally, add D_1 to $\mathcal{D}'_{\text{cand}}$. Here, $\mathcal{D}_- = \{D_3, D_4\}$. Update $\mathcal{D}_{\text{cand}} \leftarrow \mathcal{D}_{\text{cand}} \setminus \{D_1, D_3, D_4\}$, leaving $30 - 2 - 1 = 27$ candidates, and decrement d_{left} to 4. Repeat the procedure.

Algorithm 1 Phase 2: Fine Selection via Group Testing

Input: Candidate set $\mathcal{D}_{\text{cand}}$ of attributes (with $|\mathcal{D}_{\text{cand}}| \leq d_{\text{int}} m$), upper bound $d_{\max}$ on d_{int}
Output: Updated set $\mathcal{D}_{\text{cand}}$ containing all key attributes (larger than d_{int} if the user quits earlier)

$d_{\text{left}} \leftarrow d_{\max}$, $\mathcal{D}'_{\text{cand}} \leftarrow \emptyset$
/*$\mathcal{D}'_{\text{cand}}$ contains identified key attributes, and $\mathcal{D}_{\text{cand}}$ contains unclassified attributes. The quantity $d_{\text{left}} = d_{\max} - |\mathcal{D}'_{\text{cand}}|$ bounds the number of remaining key attributes.*/
/*Before every question, including individual tests and binary splitting, check whether the user stops. An unanswered question leaves its attributes unclassified.*/
while $d_{\text{left}} > 0$ and $\mathcal{D}_{\text{cand}} \neq \emptyset$ **do**
  if user quits answering **then** break
  if $|\mathcal{D}_{\text{cand}}| \leq 2d_{\text{left}} - 2$ **then** ▷ Base case
    for each attribute $D_i \in \mathcal{D}_{\text{cand}}$ **do**
      if the user stops **then**
        return $\mathcal{D}_{\text{cand}} \cup \mathcal{D}'_{\text{cand}}$
      $\mathcal{D} \leftarrow \{D_i\}$
      Present s tuples from $X_{\mathcal{D}}$
      if the user stops without answering **then**
        return $\mathcal{D}_{\text{cand}} \cup \mathcal{D}'_{\text{cand}}$
      if user selects a tuple **then**
        $\mathcal{D}'_{\text{cand}} \leftarrow \mathcal{D}'_{\text{cand}} \cup \{D_i\}$, $d_{\text{left}} \leftarrow d_{\text{left}} - 1$
      $\mathcal{D}_{\text{cand}} \leftarrow \mathcal{D}_{\text{cand}} \setminus \{D_i\}$
      if $d_{\text{left}} = 0$ **then** break
  else ▷ Group testing
    $l \leftarrow |\mathcal{D}_{\text{cand}}| - d_{\text{left}} + 1$, $\alpha \leftarrow \lfloor \log_2(l/d_{\text{left}}) \rfloor$
    $g \leftarrow \min\{2^\alpha, m\}$
    Randomly sample $\mathcal{D} \subseteq \mathcal{D}_{\text{cand}}$, $|\mathcal{D}| = g$
    Present s tuples from $X_{\mathcal{D}}$
    if the user stops without answering **then**
      return $\mathcal{D}_{\text{cand}} \cup \mathcal{D}'_{\text{cand}}$
    if user selects a tuple **then**
      Apply balanced binary splitting to $\mathcal{D}$ until one key attribute a^* is isolated or the user stops; let $\mathcal{D}_-$ contain the non-key attributes inferred from completed tests
      $\mathcal{D}_{\text{cand}} \leftarrow \mathcal{D}_{\text{cand}} \setminus \mathcal{D}_-$
      if splitting stops before a key attribute is isolated **then**
        return $\mathcal{D}_{\text{cand}} \cup \mathcal{D}'_{\text{cand}}$
      $\mathcal{D}'_{\text{cand}} \leftarrow \mathcal{D}'_{\text{cand}} \cup \{a^*\}$, $d_{\text{left}} \leftarrow d_{\text{left}} - 1$
      $\mathcal{D}_{\text{cand}} \leftarrow \mathcal{D}_{\text{cand}} \setminus \{a^*\}$
    else
      $\mathcal{D}_{\text{cand}} \leftarrow \mathcal{D}_{\text{cand}} \setminus \mathcal{D}$
return $\mathcal{D}_{\text{cand}} \cup \mathcal{D}'_{\text{cand}}$

Theorem IV.3. *Under error-free relevance responses, suppose*

at most r key attributes remain among N candidates. With continued feedback, standard generalized binary splitting requires at most $B(N, r)$ questions, where

$$B(N,r) = \begin{cases} 0, & N = 0 \text{ or } r = 0, \\ N, & r \geq 1,\ 1 \leq N \leq 2r-2, \\ (\alpha+2)r + p - 1, & r \geq 1,\ N \geq 2r-1, \end{cases}$$

where, in the third case, $l = N - r + 1$, $\alpha = \lfloor \log_2(l/r) \rfloor$, and p, θ are defined by the unique decomposition

$$l = 2^\alpha r + 2^\alpha p + \theta, \qquad 0 \leq p < r, \qquad 0 \leq \theta < 2^\alpha.$$

This bound adapts the standard generalized binary-splitting analysis [38], [40].

For FHDR, let $N = |\mathcal{D}_{\text{cand}}|$ at the start of Phase 2. A complete Phase 1 leaves $N \leq d_{\text{int}} m$ candidates. With $m = 7$ and $d_{\text{max}} = 5$, Phase 2 starts with $r = d_{\text{left}} = 5$ and $N \leq 35$. The analysis of the capped procedure covers every candidate count $0 \leq N \leq 35$ at this starting state and gives the same upper bound $B(N, 5)$.

Example IV.4. If $d_{\text{int}} = 3$, Phase 1 leaves at most 21 candidates. Since $B(21, 5) = 17$, Phase 2 requires at most 17 questions. If $d_{\text{int}} = 5$, at most 35 candidates remain and $B(35, 5) = 21$. Thus, Phase 2 requires at most 21 questions under $m = 7$, $d_{\text{max}} = 5$, and $d_{\text{int}} \leq 5$. These bounds concern completed attribute discovery and exclude Phase 3 questions.

A negative group test eliminates the tested attributes, whereas a positive test identifies one key attribute through binary splitting. Induction on the remaining problem size gives the standard bound. The detailed induction and the analysis of FHDR's capped procedure are provided in Appendix B.

The two-phase design reduces the display size before applying generalized binary splitting [39]. Applied directly to all d attributes, its first group would contain 2^α attributes, where $\alpha = \lfloor \log_2((d - d_{\text{left}} + 1)/d_{\text{left}}) \rfloor$. For $d = 100$ and $d_{\text{left}} = 5$, this gives 16 displayed attributes. Phase 1 first reduces the candidate set to at most $d_{\text{int}} m$, keeping the initial Phase 2 group within the display limit, as formalized below.

Theorem IV.5. *The size of the batch for the first group test in Phase 2 would be no more than m.*

By guaranteeing that every question involves at most m attributes, this two-phase design preserves an efficient dimension reduction process, while ensuring a user-friendly interaction.

C. Regret Minimization

After attribute discovery, FHDR applies an adapted UH-RANDOM procedure [18] over the retained attributes. With sufficient feedback, it identifies a favorite tuple. If the user stops before interactive Phase 3, ATTRIBUTESUBSET returns K tuples using the current candidate attributes.

The adapted UH-RANDOM procedure progressively refines the feasible utility region $\mathcal{R}$ over the retained attributes. Candidate tuples are retained while they can be optimal for a utility vector in this region. The procedure terminates when a favorite tuple is certified or the user stops. On early stopping, it returns a tuple maximizing utility under some $v \in \mathcal{R}$.

Algorithm 2 Overall Framework

Input: A database X, desired output size K under insufficient feedback, and an unknown utility vector u (accessed through interactive queries)
Output:
(1) Sufficient feedback: $p^* \in \arg\max_{p \in X} u \cdot p$.
(2) Stopping before Phase 3: $S \subseteq X$, $|S| = K$.

$\mathcal{D}_{\text{cand}} \leftarrow \{D_1, \ldots, D_d\}$, $\mathcal{C} \leftarrow \emptyset$ ▷ comparison constraints
Select m and s ▷ internal parameters (default: $m = 7$, $s = 2$)
while attribute discovery is incomplete and the user continues answering **do**
  Select $\mathcal{D}$ according to the current Phase 1 or Phase 2 test
  Randomly sample and display s tuples projected onto $\mathcal{D}$, and receive feedback
  if the user stops without answering **then**
    break
  Update $\mathcal{D}_{\text{cand}}$ according to the discovery rules
  Add $a_{\mathcal{D}}(p, q)$ to $\mathcal{C}$ for each elicited strict comparison
if the user stops before entering Phase 3 **then**
  return ATTRIBUTESUBSET$(X, \mathcal{D}_{\text{cand}}, K)$
Initialize the feasible utility region $\mathcal{R}$ over the retained attributes using $\mathcal{C}$
repeat
  if a tuple p^* is certified as a favorite for every utility vector in $\mathcal{R}$ **then**
    return p^*
  if the user stops **then**
    Select $v \in \mathcal{R}$
    return any $\hat{p} \in \arg\max_{p \in X} v \cdot p_{\mathcal{D}_{\text{cand}}}$
  Perform the next comparison prescribed by the adapted UH-RANDOM procedure and update $\mathcal{R}$
until a recommendation is returned

In Phases 1 and 2, relevance responses update the candidate attribute set. Tuple comparisons can also constrain relative utility weights. For tuples p, q compared on the displayed attribute set $\mathcal{D}$, define the difference vector $a_{\mathcal{D}}(p, q) \in \mathbb{R}^d$ by

$$a_{\mathcal{D}}(p,q)[i] = \begin{cases} p[i] - q[i], & D_i \in \mathcal{D}, \\ 0, & D_i \notin \mathcal{D}. \end{cases}$$

A strict preference for p over q on the displayed attributes gives $f_{\mathcal{D}}(p) > f_{\mathcal{D}}(q)$, equivalently $u \cdot a_{\mathcal{D}}(p, q) > 0$. To maintain a closed polytope represented by its vertices, we use the conservative closed halfspace

$$H_{p,q,\mathcal{D}} = \{v \in \mathbb{R}^d : v \cdot a_{\mathcal{D}}(p, q) \geq 0\}.$$

Its boundary is $v \cdot a_{\mathcal{D}}(p, q) = 0$, and the true utility vector remains feasible. Geometrically, the strict preference implies

$\theta(u, a_{\mathcal{D}}(p, q)) < 90°$. A tuple certified as a favorite throughout the maintained region is therefore also a favorite under u.

At least two key attributes must be displayed for a comparison to constrain their relative weights. If no key attribute is displayed, the user gives an irrelevant-attributes response. If exactly one key attribute i is displayed, the inequality reduces to $u[i](p[i] - q[i]) > 0$. Since $u[i] > 0$, this is equivalent to $p[i] > q[i]$ and provides no additional information about relative utility weights.

We retain these displayed-coordinate difference vectors together with their original attribute indices. When initializing UH-RANDOM, we restrict each vector to the retained candidate attributes and use the resulting constraints to initialize the feasible utility region. Under the stated response model, eliminated attributes have zero utility weight, so removing their coordinates preserves each constraint.

D. Summary

FHDR uses coarse elimination and generalized binary splitting to reduce the candidate attribute space without upfront attribute selection, then produces recommendations according to the available feedback. With sufficient feedback, FHDR identifies a favorite tuple. If interaction ends before Phase 3, ATTRIBUTESUBSET returns K recommendations; if it ends during Phase 3, FHDR returns one recommendation based on the current feasible utility region. The pseudocode for the overall algorithm is presented in Algorithm 2.

V. SINGLE-ROUND ALGORITHM

We propose ATTRIBUTESUBSET, a single-round algorithm for Problem 2 [III-C0b]. It selects K tuples from X using the current candidate attributes $\mathcal{D}_{\text{cand}}$, without further feedback.

*a) **Motivation**:* When interaction ends before Phase 3, the candidate set may be reduced but still moderately large (e.g., about 30 attributes). Single-round methods such as SPHERE [17] and CORESETHS [41] claim to be applicable to any dimensionality, but their performance degrades severely in high-dimensional settings. Take SPHERE for example, the theoretical bounds $O(ne^{O(\sqrt{d \log n})} + nk^3 d)$ [42] and $O(nkd^2 2^d + nk^3 d)$ [43] grow sub-exponentially with d. Its requirement of $k \geq d$ results in overly large recommendation sets (e.g., $k \geq 100$ when $d = 100$), which are unsuitable for real-world use. SPHERE also optimizes worst-case regret over the full utility space without exploiting sparse preferences.

*b) **Inner Runs**:* To address these limitations, we propose the ATTRIBUTESUBSET algorithm, which consists of multiple inner runs. Each inner run i independently samples a uniformly random w-element subset of $\mathcal{D}_{\text{cand}}$. We compute the skyline of the projected dataset using BSKYTREE [44] and apply SPHERE with inner output budget $k > w$, obtaining S_i. Attributes are distinct within a sample and may recur across runs. We use $w = 6$ as an experimental default, balancing execution time and regret in the parameter study (Figure 17). We choose $k = w + 1 = 7$, the smallest integer satisfying $k > w$. BSKYTREE uses balanced pivot-based partitioning to prune dominated regions and avoid comparisons between incomparable regions.

Algorithm 3 ATTRIBUTESUBSET

Input: A database X, a candidate set $\mathcal{D}_{\text{cand}}$ and the final output size K
Output: A subset $S \subseteq X$ with low regret ratio

$S \leftarrow \emptyset$
/*default value: $w = 6, k = 7, max_iter = 50$*/
Select some $w \geq d_{\max}$, $k > w$, max_iter ▷ internal parameters
if $w \geq |\mathcal{D}_{\text{cand}}|$ **then**
    Construct a new dataset $X_{\mathcal{D}_{\text{cand}}}$ restricted to dimensions in $\mathcal{D}_{\text{cand}}$
    $X_{\mathcal{D}_{\text{cand}}} = \text{SKYLINE}(X_{\mathcal{D}_{\text{cand}}})$, $S = \text{SPHERE}(X_{\mathcal{D}_{\text{cand}}}, K)$
    return S
$iter \leftarrow 0$
while $|S| < K$ and $iter < max_iter$ **do**
    Independently sample w distinct dimensions uniformly from $\mathcal{D}_{\text{cand}}$
    Construct a new dataset $X_{\mathcal{D}_w}$ restricted to the w dimensions
    $X_{\mathcal{D}_w} = \text{SKYLINE}(X_{\mathcal{D}_w})$, $S_i = \text{SPHERE}(X_{\mathcal{D}_w}, k)$
    Convert points in S_i back to the original dimension d
    $S := S \cup S_i$
    $iter \leftarrow iter + 1$
if $|S| < K$ **then**
    $S := S \cup \{K - |S| \text{ random points from } X \setminus S\}$
if $|S| > K$ **then**
    $S :=$ K random points from S
return S

*c) **Aggregation**:* Let $U = \bigcup_{i=1}^{N} S_i$. We stop when $|U| \geq K$ or max_iter inner runs have completed. If $|U| < K$, we add $K - |U|$ random tuples from $X \setminus U$; if $|U| > K$, we uniformly sample K distinct tuples from U. Denote S as the resulting set. The pseudocode is presented in Algorithm 3.

*d) **Small Candidate Sets**:* If $|\mathcal{D}_{\text{cand}}| \leq w$, we use all candidate attributes in a single skyline–SPHERE run with output budget K.

*e) **Regret Transfer and Trimming**:* The following theorem transfers an inner output's regret bound to the original dataset when all key attributes are sampled.

Theorem V.1. *Fix u, and suppose the sampled set $\mathcal{D}_w$ contains all its key attributes. Let $S_i \subseteq X$ denote the original tuples corresponding to the inner output. If its projected output has regret ratio at most ε for every nonnegative unit-L_1 utility vector over $\mathcal{D}_w$, then $\text{rr}_X(S_i, u) \leq \varepsilon$.*

Omitted attributes have zero weight, so original and projected utilities coincide. See Appendix B.

The union U inherits any regret bound satisfied by an inner output S_i, and adding tuples preserves this bound. To obtain exactly K tuples, we uniformly trim an oversized union. Since the last run adds at most k tuples, $|U| \leq K + k - 1$. For a fixed utility vector u, trimming retains a specified utility-maximizing tuple in U with probability $K/|U| \geq K/(K +$

$k-1$), preserving the union's regret bound with at least this probability. See Appendix B for a proof.

f) ***Coverage Analysis****:* Fix a user's key-attribute set $\mathcal{H} \subseteq \mathcal{D}_{\text{cand}}$. Let $D = |\mathcal{D}_{\text{cand}}|$ and $h = |\mathcal{H}| = d_{\text{int}}$. Consider the repeated-sampling branch, where $1 \le h \le w < D$. Let P_{cover} be the probability that a sample contains $\mathcal{H}$.

Lemma V.2. *Under uniform sampling,*

$$P_{cover} = \frac{\binom{D-h}{w-h}}{\binom{D}{w}} = \frac{\binom{w}{h}}{\binom{D}{h}} \ge \left(\frac{w-h+1}{D}\right)^h.$$

Exactly $\binom{D-h}{w-h}$ samples contain the fixed key-attribute set; see Appendix B.

For a fixed positive integer t, let C_t denote the event that at least one of t independent attribute samples contains every key attribute. Then

$$\Pr(C_t) = 1 - (1 - P_{\text{cover}})^t.$$

For a target coverage confidence $\gamma \in (0,1)$, the minimum prescribed number of independent samples is

$$t_\gamma = \left\lceil \frac{\ln(1-\gamma)}{\ln(1-P_{\text{cover}})} \right\rceil, \qquad 0 < P_{\text{cover}} < 1.$$

These expressions quantify coverage for a prescribed number of independent samples. ATTRIBUTESUBSET stops according to the accumulated union size and the iteration cap.

In the reported Car example, 12 interaction questions leave $D = 12$ candidate attributes, with $h = 3$ and $w = 6$. ATTRIBUTESUBSET completes 33 inner runs in approximately 0.1 s. For a prescribed sequence of 33 independent samples, the coverage probability is $1 - (10/11)^{33} \approx 95.7\%$.

Overall, the ATTRIBUTESUBSET algorithm effectively bridges the gap between the general applicability of SPHERE and the practical constraints of high-dimensional regret minimization with sparse user preferences, introducing both efficiency and accuracy.

VI. EXPERIMENTS

We conducted experiments with an Apple M2 Pro chip and 32GB of RAM. All programs were implemented in C/C++.

a) ***Datasets.*** *:* We generate synthetic datasets with independent, uniformly distributed attributes using the skyline data generator [45].

We also evaluate five high-dimensional real datasets: NBA [46], with 4,790 player-season tuples and 104 attributes; Car [47], with 5,500 vehicles and 57 attributes; House [48], with 15,171 properties and 33 attributes; Energy [49], with 36,043 wave-energy farm configurations and 149 attributes; and Activity [50], with 2,872,533 processed activity profiles and 54 attributes. Detailed statistics appear in Appendix E.

We encoded missing values using the minimum observed value of the corresponding attribute. We removed constant attributes and normalized the remaining attributes to $[0,1]$, reversing those for which smaller values are preferred, as described in Section III. All reported dimensionalities refer to the retained attributes.

b) ***Algorithms.*** *:* We compare FHDR with two baselines: (1) UTILITYAPPROX [2], an interactive method applicable to high-dimensional data; and (2) SPHERE-ADAPT, an adaptation of the single-round SPHERE algorithm [17]. For the set-output experiments, we adapt UTILITYAPPROX to return the K highest-scoring tuples under its estimated utility vector. To ensure fairness, SPHERE-ADAPT receives the candidate attribute set obtained by FHDR's dimension reduction process and runs SPHERE on the corresponding projected data. This comparison evaluates ATTRIBUTESUBSET against SPHERE within the same discovered attribute space.

Table I summarizes the practical limitations of additional methods. In the tested high-dimensional settings, UH-RANDOM/UH-SIMPLEX and HD-PI encounter convex-hull computation failures, while VERIFY-POINT encounters Qhull errors and excessive memory use. BREAKPOINT requires a counter repair on the large synthetic input and incurs substantial interaction and output costs. Detailed reproduction results are provided in Appendix C. We retain UTILITYAPPROX and SPHERE-ADAPT for quantitative comparison and evaluate UH-RANDOM's upfront attribute-selection mechanism in the user study in Section VI-C.

c) ***Parameter Setting.*** *:* We evaluate each algorithm by varying five key parameters: (1) dataset size n, (2) dimensionality d, (3) number of key attributes d_{int}, (4) number of answered questions q, and (5) output size K for Problem 2 [III-C0b].

Unless otherwise specified, we use $d_{\text{int}} = 3$ [19], [20], [23]–[26], question size $s = 2$ [2], question budget $q = 15$ (see [VI-A2] for justification), output size $K = 30$ [17], and target regret ratio $\varepsilon = 0$ for UH-RANDOM. For synthetic datasets, we set $n = 10^5$ [2], [17], [18], [31] and $d = 100$.

We study two cases: complete interaction and limited feedback, with $q = 15$ or $q = 0$. We also vary the maximum number of displayed attributes m and the number sampled per ATTRIBUTESUBSET inner run w on real datasets across different d_{int}. Based on these results, we use $m = 7$ and $w = 6$; further details appear in Appendix E.

d) ***Performance Measurement.*** *:* We evaluate the algorithms using four metrics: (1) *execution time*, the end-to-end time (excluding user response time) for the final output; (2) *regret ratio*, the relative utility loss between the best returned tuple and the global optimum under the ground-truth utility function; (3) *outperformance rate*, the fraction of repeated runs in which our algorithm achieves the lowest regret ratio, with ties broken by execution time; and (4) *number of questions*, which measures user effort.

Each experiment is repeated 100 times with independently generated utility functions, and we report the averages.

A. Results on Synthetic Datasets

We evaluate sufficient- and limited-feedback settings separately. All experimental curves labeled SPHERE correspond to SPHERE-ADAPT, as defined above.

1) Problem 1: [III-C0a]

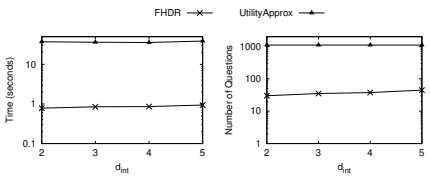


Fig. 2: P1: Vary d_{int}

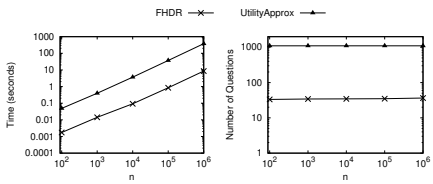


Fig. 3: P1: Vary n

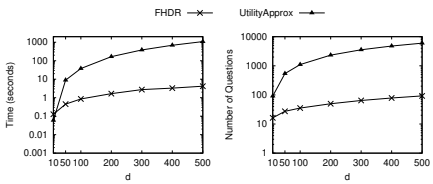


Fig. 4: P1: Vary d

We first evaluate favorite-tuple retrieval under sufficient feedback, reporting the number of questions and execution time. FHDR terminates when its recommendation is certified as a favorite tuple. For UTILITYAPPROX, we set $\varepsilon = 0$ and terminate when $\sum_{i=1}^{d}(U_i - L_i) < 10^{-9}$ or the 10,000-question cap is reached, where L_i and U_i are its utility-coefficient bounds. Its reported costs correspond to this numerical convergence criterion. We compare with UTILITYAPPROX here and evaluate the single-round SPHERE-ADAPT in the limited-feedback setting.

We vary the number of key attributes d_{int} from 2 to 5, following the preference-sparsity setting motivated above. As shown in Figure 2, the number of questions required by our algorithm increases moderately from 29 to 42, whereas UTILITYAPPROX consistently requires approximately 1100 questions, making it impractical for real-world use. The execution time remains low, increasing only from 0.02 s to 0.05 s, while UTILITYAPPROX requires approximately 37 s. The nearly constant behavior of UTILITYAPPROX follows from its weakly truthful design: it refines the hidden utility function using synthetic tuples rather than actual data points. Consequently, for a fixed dimensionality d, the number of questions is largely independent of the input dataset. With a fixed number of tuples, its execution time therefore also remains nearly constant.

Figures 3 and 4 evaluate scalability with respect to the number of tuples n and the dimensionality d. As shown in Figure 3, our algorithm scales well with n, completing within a few seconds even when $n = 10^6$. In contrast, the execution time of UTILITYAPPROX grows approximately linearly with n, reaching 378 s at $n = 10^6$. For fixed d, our algorithm requires approximately 34 questions regardless of n, whereas UTILITYAPPROX requires about 1100. As d increases from 10 to 500, FHDR's question count grows much more slowly than UTILITYAPPROX's. By comparison, UTILITYAPPROX requires 91 questions at $d = 10$ and 5989 questions at $d = 500$. The execution-time results exhibit a similar trend.

2) Problem 2: [III-C0b]

We next evaluate recommendation quality under fixed question budgets.

Figure 5 varies q from 15 to 36 in increments of 3. Under the default settings, $15 = \lceil 100/7 \rceil$ questions complete Phase 1 and leave at most $d_{\text{int}}m = 21 \leq K = 30$ candidate attributes, allowing comparison with SPHERE-ADAPT under the same output budget.

Figure 5 shows FHDR reaching zero regret with sufficient Phase 3 feedback. Based on aforementioned algorithm design, if the user stops before Phase 3, FHDR returns a recommendation set; if interaction continues into Phase 3, FHDR returns one tuple and progressively refines the recommendation. Before FHDR enters the interactive Phase 3 branch, all methods return $K = 30$ tuples. Afterward, FHDR and UTILITYAPPROX return one tuple, while SPHERE-ADAPT uses the output budget $K = sq'$, where q' counts FHDR's additional Phase 3 questions and s is the number of tuples displayed per question. Following the tuple-presentation accounting in [18], this budget gives the single-round method a corresponding allowance for FHDR's additional interaction. UTILITYAPPROX's regret increase coincides with its output shrinking from $K = 30$ tuples to one, as shown in the output-size panel. Returning fewer tuples can increase regret despite additional feedback; the curve therefore reflects changes in both the question budget and output size. Our method consistently achieves a lower regret ratio compared with UTILITYAPPROX. While SPHERE-ADAPT attains similar regret ratios, our method performs better in over 60% of cases.

We vary the output size K in Figure 6. FHDR achieves lower regret ratios than UTILITYAPPROX. Its execution time increases from 1 to 3 seconds, compared with ~ 11 seconds for SPHERE-ADAPT. Although slightly slower than UTILITYAPPROX, it provides substantially better result quality and achieves the lowest regret ratio in $\sim 60\%$ of the runs.

Figure 7 shows the effect of varying d_{int}, which reflects increasing preference complexity. For all three algorithms, the regret ratio grows approximately linearly as d_{int} increases from 2 to 5, with lower regret observed at smaller values. Execution times remain consistent with earlier results, although our algorithm becomes slightly faster as d_{int} increases. With a fixed question budget, a larger d_{int} produces a larger candidate set $|\mathcal{D}_{\text{cand}}|$, reducing overlap among the inner output sets S_i. The union can therefore reach the fixed output size K in fewer inner runs, reducing execution time.

Figures 8 and 9 evaluate scalability with respect to the number of tuples n and dimensionality d. As n increases, our algorithm maintains a regret ratio below 0.045, substantially lower than UTILITYAPPROX, whose regret increases noticeably. FHDR's runtime grows approximately linearly with n, whereas SPHERE-ADAPT scales less favorably. Section V discusses SPHERE's dimension-dependent cost.

As shown in Figure 9, our algorithm also consistently outperforms UTILITYAPPROX in regret ratio and remains close to SPHERE-ADAPT. Its runtime stays within a few seconds, first peaking at $d = 50$, then decreasing slightly before rising again. The temporary decrease occurs because a larger d produces

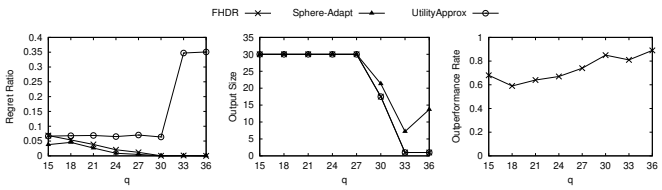


Fig. 5: P2: Vary q

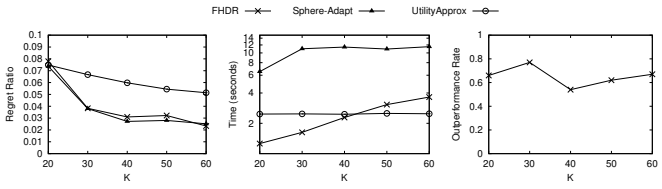


Fig. 6: P2: Vary K

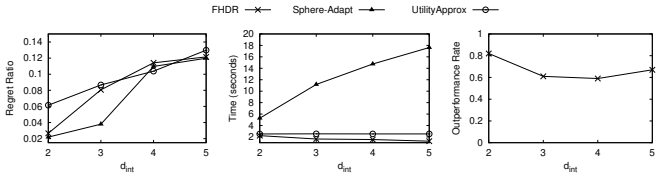


Fig. 7: P2: Vary d_{int}

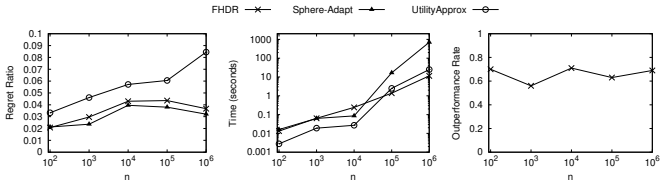


Fig. 8: P2: Vary n

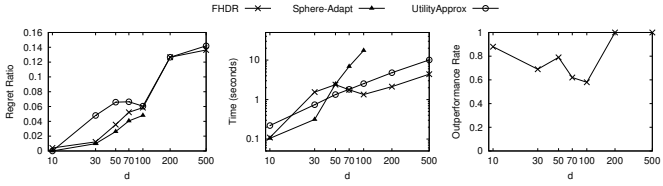


Fig. 9: P2: Vary d

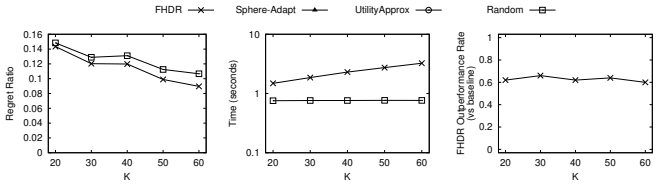


Fig. 10: $P2^*$: Vary K

a larger candidate set $|\mathcal{D}_{\text{cand}}|$ under a fixed question budget, reducing overlap among the inner output sets and hence the number of SPHERE runs within ATTRIBUTESUBSET. At larger d, dataset-loading costs begin to dominate. SPHERE-ADAPT becomes infeasible beyond $d = 100$ under the default K and q settings, whereas our method continues to return valid results in approximately one second, demonstrating the robustness in extremely high-dimensional settings.

Zero-Feedback Evaluation ($P2^*$): We also evaluate the special case of zero-feedback in Problem 2, denoted by $P2^*$, in which the user answers no questions ($q = 0$). FHDR directly invokes ATTRIBUTESUBSET on all original attributes, without attribute elimination or elicited comparison constraints. We compare our method with RANDOM, a baseline which samples K distinct tuples uniformly without replacement from the entire dataset.

Figure 10 shows that FHDR achieves lower mean regret than RANDOM at every tested K. As K increases from 20 to 60, FHDR's mean regret decreases from 0.1434 to 0.0897, compared with 0.1483 to 0.1067 for RANDOM. FHDR achieves lower or equal regret in 60–66% of runs.

B. Results on Real Datasets

We report NBA results for sufficient and limited feedback; results for the other real datasets appear in Appendix D.

For the case where the user answers all questions (Problem 1 (III-C0a)), as d_{int} increases from 2 to 5, FHDR requires 30–40 questions and approximately 0.04 s, while UtilityApprox requires 1,149 questions (Figure 11a).

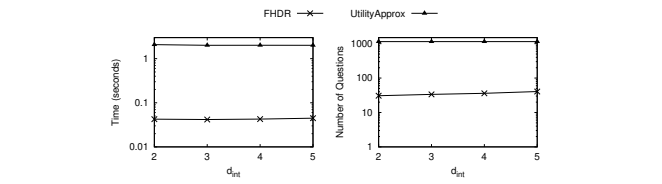
Figures 11b–11e report results for Problem 2, including its zero-feedback special case. For the positive-feedback experiments, we set the default budget to $q = \lceil d/m \rceil$, sufficient to complete Phase 1, following [51] and the synthetic-data setting. For NBA, this gives $q = 15$. When varying q, we consider values from $\lceil d/m \rceil$ to the maximum number of questions observed across repeated runs for identifying the user's favorite tuple.

Figure 11b evaluates the effect of varying d_{int}. FHDR achieves regret ratios comparable to SPHERE-ADAPT and lower than UTILITYAPPROX. Its execution time decreases slightly from 0.08 to 0.05 seconds and remains lower than both baselines. On NBA, the smaller tuple count makes the absolute runtime difference less pronounced.

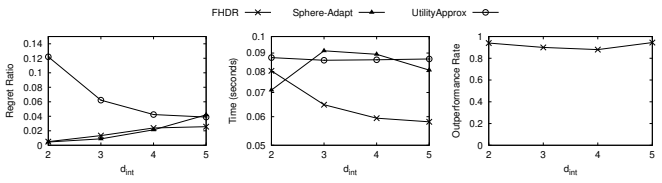
As K increases, regret decreases for all methods (Figure 11c). AttributeSubset achieves an outperformance rate of approximately 0.9 and runs faster than Sphere-Adapt. FHDR maintains low regret with only 15 questions and reaches zero regret with sufficient feedback (Figure 11d). UTILITYAPPROX's regret increase again coincides with the reduction to singleton output under the evaluation protocol described above.

Zero-feedback special case ($P2^*$): As shown in Figure 11e, FHDR's mean regret is lower than RANDOM by an order of magnitude. The quality advantage is greater than on the synthetic dataset: FHDR achieves lower or equal regret in 96–100% of runs, with execution time all below 0.1s. Thus, the zero-feedback advantage is more pronounced on NBA, with a small additional computation cost.

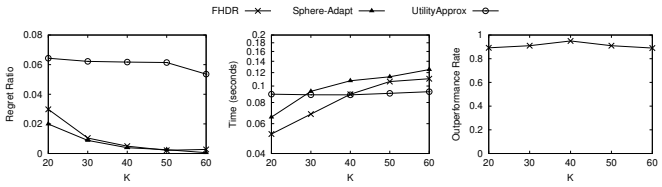
Across the evaluated real datasets, FHDR combines low regret with efficient execution and reduced interaction cost, consistent with the synthetic-data results. These findings support its practical effectiveness on high-dimensional real data.



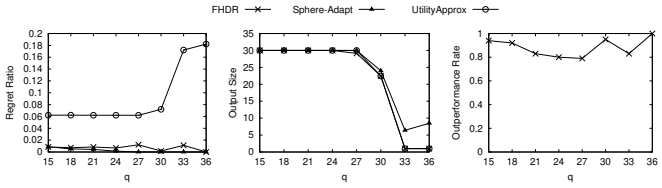
(a) P1: Vary d_{int}



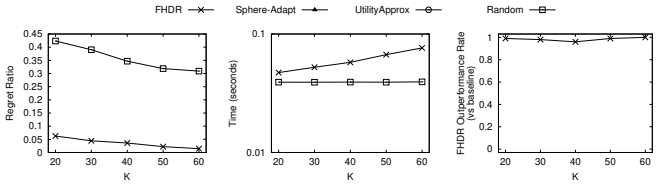
(b) P2: Vary d_{int}



(c) P2: Vary K

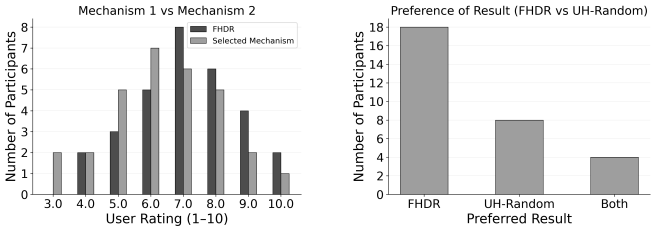


(d) P2: Vary q



(e) $P2^*$: Vary K

Fig. 11: Results on *NBA* dataset



(a) Mechanism 1 vs. 2

(b) Result preference

Fig. 12: User study results

C. User Study

We conducted a user study[1] on the Car dataset to evaluate the practical usability of FHDR, because vehicle selection is intuitive and the dataset's 57 attributes provide a representative high-dimensional setting. The study involved the same 30 participants as the attribute-selection survey, and the results were averaged across participants.

We compare FHDR with UH-RANDOM, evaluating their interaction mechanisms and returned recommendations. UTILITYAPPROX is excluded because it requires over 1,000 questions, making a direct user-study comparison impractical. Single-round methods such as SPHERE are excluded from this interaction-experience comparison. To reduce bias, the interface does not reveal which method is ours.

Participants interact with both methods. During each tuple comparison, the interface displays only the attributes used in that question and their values. Participants are explicitly instructed to treat omitted attributes as identical, following the interaction model in Section III. For UH-RANDOM, they select between 2 and 7 relevant attributes and interact with the method on the projected dataset. They then compare the recommendations and interaction mechanisms. Mechanism 1, used by FHDR, relies on pairwise comparisons over a small number of displayed attributes. Mechanism 2, used by UH-RANDOM, requires upfront attribute selection but generally fewer subsequent questions.

Figure 12 summarizes the results, with detailed statistics in Appendix F. Figure 12a shows that Mechanism 1 received a higher average rating than Mechanism 2, with scores of 7.10 and 6.37, respectively, while Mechanism 2 exhibited greater variation. These results support the usability advantage of avoiding upfront attribute selection. In Figure 12b, 18 participants preferred FHDR's output, 8 preferred UH-Random's, and 4 rated them as comparable.

VII. CONCLUSION

We presented FHDR for high-dimensional regret minimization under sparse linear preferences. FHDR discovers key attributes without upfront attribute selection and supports flexible interaction, including skipped questions, early stopping, and zero feedback. It identifies a favorite tuple with sufficient feedback and provides compact recommendations when feedback is limited. Experiments on synthetic and real datasets demonstrate low regret, efficient execution, and substantially reduced interaction cost, while the user study supports its practical usability. Future work could investigate error-tolerant interaction to handle inconsistent or unreliable responses.

[1]Study interface: https://highdim-rm-demo-production.up.railway.app/.

REFERENCES

[1] Y.-C. Chang, L. Bergman, V. Castelli, C.-S. Li, M.-L. Lo, and J. R. Smith, "The onion technique: Indexing for linear optimization queries," in *Proceedings of ACM SIGMOD International Conference on Management of Data*, 2000, pp. 391–402.

[2] D. Nanongkai, A. Lall, A. Das Sarma, and K. Makino, "Interactive regret minimization," in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, 2012, pp. 109–120.

[3] D. Nanongkai, A. Das Sarma, A. Lall, R. J. Lipton, and J. Xu, "Regret-minimizing representative databases," *Proceedings of the VLDB Endowment*, vol. 3, no. 1–2, pp. 1114–1124, 2010.

[4] M. Goncalves and M.-E. Vidal, "Top-k skyline: a unified approach," in *On the Move to Meaningful Internet Systems 2005: OTM 2005 Workshops*, 2005, pp. 790–799.

[5] A. N. Papadopoulos, A. Lyritsis, A. Nanopoulos, and Y. Manolopoulos, "Domination mining and querying," in *Data Warehousing and Knowledge Discovery*, ser. Lecture Notes in Computer Science, vol. 4654, 2007, pp. 145–156.

[6] M. A. Soliman, I. F. Ilyas, and K. C.-C. Chang, "Top-k query processing in uncertain databases," in *2007 IEEE 23rd International Conference on Data Engineering*. IEEE, 2007, pp. 896–905.

[7] J. Lee, G.-w. You, and S.-w. Hwang, "Personalized top-k skyline queries in high-dimensional space," *Information Systems*, vol. 34, no. 1, pp. 45–61, 2009.

[8] X. Lian and L. Chen, "Top-k dominating queries in uncertain databases," in *Proceedings of the 12th International Conference on Extending Database Technology: Advances in Database Technology*, 2009, pp. 660–671.

[9] C.-Y. Chan, H. V. Jagadish, K.-L. Tan, A. K. H. Tung, and Z. Zhang, "Finding k-dominant skylines in high dimensional space," in *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*, 2006, pp. 503–514.

[10] ——, "On high dimensional skylines," in *Advances in Database Technology - EDBT 2006*. Springer, 2006, pp. 478–495.

[11] X. Lin, Y. Yuan, Q. Zhang, and Y. Zhang, "Selecting stars: The k most representative skyline operator," in *2007 IEEE 23rd International Conference on Data Engineering*, 2007, pp. 86–95.

[12] D. Mindolin and J. Chomicki, "Discovering relative importance of skyline attributes," *Proceedings of the VLDB Endowment*, vol. 2, no. 1, pp. 610–621, 2009.

[13] D. Papadias, Y. Tao, G. Fu, and B. Seeger, "Progressive skyline computation in database systems," *ACM Transactions on Database Systems*, vol. 30, no. 1, pp. 41–82, 2005.

[14] P. K. Agarwal, N. Kumar, S. Sintos, and S. Suri, "Efficient algorithms for k-regret minimizing sets," in *16th International Symposium on Experimental Algorithms (SEA 2017)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 75. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017, pp. 7:1–7:23.

[15] A. Asudeh, A. Nazi, N. Zhang, and G. Das, "Efficient computation of regret-ratio minimizing set: A compact maxima representative," in *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 2017, pp. 821–834.

[16] P. Peng and R. C.-W. Wong, "Geometry approach for k-regret query," in *2014 IEEE 30th International Conference on Data Engineering*, 2014, pp. 772–783.

[17] M. Xie, R. C.-W. Wong, J. Li, C. Long, and A. Lall, "Efficient k-regret query algorithm with restriction-free bound for any dimensionality," in *Proceedings of the 2018 ACM International Conference on Management of Data*. ACM, 2018, pp. 959–974.

[18] M. Xie, R. C.-W. Wong, and A. Lall, "Strongly truthful interactive regret minimization," in *Proceedings of the 2019 International Conference on Management of Data*, ser. SIGMOD '19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 281–298.

[19] G. Gigerenzer and D. G. Goldstein, "Reasoning the fast and frugal way: models of bounded rationality." *Psychological Review*, vol. 103, no. 4, pp. 650–669, 1996.

[20] A. Graefe and J. S. Armstrong, "Predicting elections from the most important issue: A test of the take-the-best heuristic," *Journal of Behavioral Decision Making*, vol. 25, no. 1, pp. 41–48, 2012. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/bdm.710

[21] L. Martignon and U. Hoffrage, "Why does one-reason decision making work? a case study in ecological rationality," in *Simple heuristics that make us smart*, G. Gigerenzer, P. M. Todd, and the ABC Research Group, Eds. Oxford University Press, 1999, pp. 119–140.

[22] G. Gigerenzer and P. M. Todd, "Fast and frugal heuristics: The adaptive toolbox," in *Simple heuristics that make us smart*, G. Gigerenzer, P. M. Todd, and the ABC Research Group, Eds. Oxford University Press, 1999, pp. 3–34. [Online]. Available: https://pure.mpg.de/rest/items/item_2102905/component/file_2102904/content

[23] L. Wang, Y. Xu, H. Lee, and A. Li, "Preferred product attributes for sustainable outdoor apparel: A conjoint analysis approach," *Sustainable Production and Consumption*, vol. 29, pp. 657–671, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352550921003249

[24] L. C. Schaupp and F. Bélanger, "A conjoint analysis of online consumer satisfaction," *Journal of Electronic Commerce Research*, vol. 6, no. 2, pp. 95–111, 2005. [Online]. Available: https://www.jecr.org/sites/default/files/06_2_p02_0.pdf

[25] K. Tano, M. Kamuanga, M. D. Faminow, and B. Swallow, "Using conjoint analysis to estimate farmer's preferences for cattle traits in west africa," *Ecological Economics*, vol. 45, no. 3, pp. 393–407, 2003, valuing Animal Genetic Resources. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921800903000934

[26] A. Dunnett, J. Moorhouse, C. Walsh, and C. Barry, "Choosing a university: A conjoint analysis of the impact of higher fees on students applying for university in 2012," *Tertiary Education and Management*, vol. 18, no. 3, pp. 199–220, 2012. [Online]. Available: https://doi.org/10.1080/13583883.2012.657228

[27] W. Cao, J. Li, H. Wang, K. Wang, R. Wang, R. C.-W. Wong, and W. Zhan, "k-regret minimizing set: Efficient algorithms and hardness," in *20th International Conference on Database Theory (ICDT 2017)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 68. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017, pp. 11:1–11:19.

[28] S. Chester, A. Thomo, S. Venkatesh, and S. Whitesides, "Computing k-regret minimizing sets," *Proceedings of the VLDB Endowment*, vol. 7, no. 5, pp. 389–400, 2014.

[29] K. G. Jamieson and R. Nowak, "Active ranking using pairwise comparisons," in *Advances in Neural Information Processing Systems*, vol. 24. Curran Associates, Inc., 2011. [Online]. Available: https://papers.nips.cc/paper_files/paper/2011/hash/6c14da109e294d1e8155be8aa4b1ce8e-Abstract.html

[30] L. Qian, J. Gao, and H. V. Jagadish, "Learning user preferences by adaptive pairwise comparison," *Proceedings of the VLDB Endowment*, vol. 8, no. 11, pp. 1322–1333, 2015.

[31] W. Wang, R. C.-W. Wong, and M. Xie, "Interactive search for one of the top-k," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD/PODS '21, 2021, pp. 1920–1932.

[32] Q. Chen and R. C.-W. Wong, "Finding best tuple via error-prone user interaction," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 1803–1816.

[33] L. Do, O. Kim, C. Chai, and A. Lall, "The power of two: Simplified user interaction for the indistinguishability query," in *Proceedings of the 37th International Conference on Scalable Scientific Data Management*, ser. SSDBM '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3733723.3733725

[34] N. Cowan, "The magical number 4 in short-term memory: A reconsideration of mental storage capacity," *Behavioral and Brain Sciences*, vol. 24, no. 1, pp. 87–114, 2001.

[35] P. Slovic, "The construction of preference," *American Psychologist*, vol. 50, no. 5, pp. 364–371, 1995.

[36] J. Meyerhoff and M. Oehlmann, "The performance of full versus partial profile choice set designs in environmental valuation," *Ecological Economics*, vol. 204, p. 107665, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921800922003263

[37] A. H. Y. Chan, M. Tao, S. Marsh, and H. Petousis-Harris, "Vaccine decision making in New Zealand: a discrete choice experiment," *BMC Public Health*, vol. 24, no. 1, p. 447, 2024.

[38] F. K. Hwang, "A method for detecting all defective members in a population by group testing," *Journal of the American Statistical Association*, vol. 67, no. 339, pp. 605–608, 1972.

[39] Wikipedia contributors, "Group testing — wikipedia, the free encyclopedia," accessed: 2026-10-28. [Online]. Available: https://en.wikipedia.org/wiki/Group_testing#Generalised_binary-splitting_algorithm

[40] D.-Z. Du and F. K. Hwang, *Combinatorial Group Testing and Its Applications*, 2nd ed. Singapore: World Scientific, 1999. [Online]. Available: https://personal.utdallas.edu/~dzdu/book.html

[41] N. Kumar and S. Sintos, "Faster approximation algorithm for the k-regret minimizing set and related problems," in *2018 Proceedings of the Twentieth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 2018, pp. 62–74.

[42] B. Gärtner, "A subexponential algorithm for abstract optimization problems," *SIAM Journal on Computing*, vol. 24, no. 5, pp. 1018–1035, 1995.

[43] M. Sharir and E. Welzl, "A combinatorial bound for linear programming and related problems," in *Proceedings of 9th Annual Symposium on Theoretical Aspects of Computer Science (STACS)*, ser. Lecture Notes in Computer Science, vol. 577. Springer, 1992.

[44] J. Lee and S.-w. Hwang, "BSkyTree: scalable skyline computation using a balanced pivot selection," in *Proceedings of the 13th International Conference on Extending Database Technology*, ser. EDBT '10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 195–206. [Online]. Available: https://doi.org/10.1145/1739041.1739067

[45] S. Börzsönyi, D. Kossmann, and K. Stocker, "The skyline operator," in *Proceedings. 17th International Conference on Data Engineering*, 2001, pp. 421–430.

[46] Sports Reference LLC, "Basketball-reference.com," accessed: 2026-10-28. [Online]. Available: https://www.basketball-reference.com/

[47] United States Environmental Protection Agency, "Explore the automotive trends data," accessed: 2026-10-28. [Online]. Available: https://www.epa.gov/automotive-trends/explore-automotive-trends-data

[48] E. Pierce, "Austin, tx house listings," Kaggle, 2021, accessed: 2026-10-28. [Online]. Available: https://www.kaggle.com/datasets/ericpierce/austinhousingprices

[49] M. Neshat, B. Alexander, N. Sergiienko, and M. Wagner, "Large-scale wave energy farm," UCI Machine Learning Repository, 2020. [Online]. Available: https://doi.org/10.24432/C5GG7Q

[50] A. Reiss, "PAMAP2 Physical Activity Monitoring," UCI Machine Learning Repository, 2012. [Online]. Available: https://doi.org/10.24432/C5NW2H

[51] A. Lall, "The indistinguishability query," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 475–487.

[52] G. A. Miller, "The magical number seven, plus or minus two: some limits on our capacity for processing information." *Psychological Review*, vol. 63, no. 2, pp. 81–97, 1956.

[53] S. Morra, P. Patella, and L. Muscella, "Modelling working memory capacity: Is the magical number four, seven, or does it depend on what you are counting?" *Journal of Cognition*, vol. 7, no. 1, p. 60, 2024.

[54] P. R. McCullough, "A user's guide to conjoint analysis," *Marketing Research*, May 2007. [Online]. Available: https://www.macroinc.com/english/publications/a-users-guide-to-conjoint-analysis/

[55] A. E. Oltman, "Qualitative and quantitative research methods to measure consumer valuation and consumer acceptability," Master's thesis, North Carolina State University, Raleigh, NC, 2015, master of Science, Food Science. [Online]. Available: https://repository.lib.ncsu.edu/bitstreams/331d7871-e196-4b5e-81c5-95e3c600b81d/download

[56] P. E. Green and V. Srinivasan, "Conjoint analysis in consumer research: Issues and outlook," *Journal of Consumer Research*, vol. 5, no. 2, pp. 103–123, 1978.

APPENDIX A
SUMMARY OF NOTATIONS

We summarize the notations that are frequently used in Table III.

Notation	Meaning
X	The set of d-dimensional points with $\|X\| = n$
$X_{\mathcal{D}}$	X restricted to a selected subset $\mathcal{D}$ of dimensions
f and u	$f(p) = u \cdot p$ where $\sum_{i=1}^{d} u[i] = 1$
$f_{\mathcal{D}}(p)$	Partial utility, $f_{\mathcal{D}}(p) = \sum_{i:D_i \in \mathcal{D}} u[i]p[i]$
$\mathrm{rr}_X(S, u)$	The regret ratio of S over X w.r.t. u
d	The number of dimensions of the original dataset
d_{int}	The number of dimensions that the user is interested in
d_{max}	The maximum number of dimensions that a user may be interested in
d_{left}	The remaining upper bound on the number of key attributes in Phase 2
$\mathcal{D}_{\text{cand}}$	Candidate set: the set of dimensions which are not determined as *non-key*
D_i	The i^{th} dimension from the original dataset, $i \in [1, d]$
p^*	The point that a user like the most (having the maximum utility)
S	The returned subset of points with $\|S\| = K$
K	The desired final output size for Problem 2
S_i	The returned regret minimizing set of SPHERE in the i^{th} iteration with $\|S_i\| = k$
k	The inner output size of SPHERE in ATTRIBUTESUBSET
m	The maximum number of attributes displayed per question
w	The number of dimensions sampled in each iteration, in ATTRIBUTESUBSET
N	The number of iterations executed in ATTRIBUTESUBSET
s	The number of points displayed per question
ε	The target regret ratio (in UH-RANDOM)

TABLE III: Summary of Frequently Used Notations

APPENDIX B
REMAINING PROOFS

Proof of Theorem IV.3. Let $Q(N, r)$ denote the worst-case number of remaining questions for standard generalized binary splitting when at most r key attributes remain. We prove $Q(N, r) \le B(N, r)$ by strong induction on $N + r$.

First, $B(N, r)$ is nondecreasing in N for fixed r. Within each interval having fixed α, this follows from $p = \lfloor l/2^{\alpha} \rfloor - r$; at an exponent transition, the bound increases by one. Moreover, in the third branch,

$$N - B(N, r) = (2^{\alpha} - \alpha - 1)r + (2^{\alpha} - 1)p + \theta \ge 0,$$

so $B(N, r) \le N$.

If $N = 0$ or $r = 0$, no further question is required. If $1 \le N \le 2r-2$, individual testing uses at most $N = B(N, r)$ questions.

If $r = 1$, then $B(N, 1) = \lfloor \log_2 N \rfloor + 1 = \alpha + 1$. A positive group test followed by binary splitting uses at most $1 + \alpha$ questions and identifies the only possible remaining key attribute. A negative test leaves $N - 2^{\alpha} < 2^{\alpha}$ candidates; induction bounds the total by $1 + \alpha$.

If $r \ge 2$ and $2r - 1 \le N \le 3r - 2$, then $\alpha = 0$ and $B(N, r) = N$. The first question tests one attribute. Either response leaves at most $N - 1$ candidates, so induction and $B(N', r') \le N'$ bound the total by N.

It remains to consider $r \ge 2$ and $N \ge 3r - 1$. Set $b = 2^{\alpha}$. A negative response costs one question and leaves $(N - b, r)$. A positive response costs one group question and at most α splitting questions, identifies one key attribute, and leaves at most $N - 1$ candidates with remaining upper bound $r - 1$. Therefore,

$$Q(N, r) \le \max\{1 + B(N - b, r),\ 1 + \alpha + B(N - 1, r - 1)\}.$$

Substitution of the corrected decomposition gives

$$1 + B(N - b, r) = \begin{cases} B(N, r) - 1, & p = 0 \text{ and } \theta < b/2, \\ B(N, r), & \text{otherwise}, \end{cases}$$

and

$$1 + \alpha + B(N - 1, r - 1) = \begin{cases} B(N, r) - 1, & p = r - 1, \\ B(N, r), & \text{otherwise}. \end{cases}$$

Both terms are at most $B(N, r)$, completing the induction. □

a) Capped FHDR procedure: Fix $m = 7$. Define $C(0, r) = C(N, 0) = 0$, and $C(N, r) = N$ when $r \ge 1$ and $1 \le N \le 2r - 2$. Otherwise, let $g = \min\{2^{\alpha}, 7\}$ and define

$$\begin{aligned} C(N, r) &= \max\{C_-(N, r), C_+(N, r)\}, \\ C_-(N, r) &= 1 + C(N - g, r), \\ C_+(N, r) &= 1 + \lceil \log_2 g \rceil \\ &\quad + \max_{1 \le j \le g} C(N - j, r - 1). \end{aligned}$$

A negative response removes all g tested candidates. A positive response requires at most $\lceil \log_2 g \rceil$ additional questions under balanced binary splitting and removes between one and g candidates, including the identified key attribute. Thus, the two terms bound all possible responses. Every recursive state has fewer candidates, so strong induction on N establishes C as an upper bound for the capped procedure.

b) Finite verification of the FHDR bound: Evaluate the recurrence in increasing N for $0 \le N \le 35$ and $0 \le r \le 5$. All dependencies refer to previously computed rows. The resulting $r = 5$ column is

$$C(N, 5) = \begin{cases} N, & 0 \le N \le 13, \\ 7 + \lfloor N/2 \rfloor, & 14 \le N \le 23, \\ 13 + \lfloor N/4 \rfloor, & 24 \le N \le 35. \end{cases}$$

These values equal $B(N,5)$ throughout the stated range. Consequently, $C(N,5) \le 17$ for $N \le 21$, and $C(N,5) \le 21$ for $N \le 35$. This establishes the stated FHDR bounds without applying the uncapped theorem to intermediate capped states.

c) Proof of Theorem IV.5:

$$\begin{aligned}\text{batch size} &= 2^{\alpha} = 2^{\lfloor \log_2((|\mathcal{D}_{\text{cand}}| - d_{\text{left}} + 1)/d_{\text{left}}) \rfloor} \\ &= 2^{\lfloor \log_2((|\mathcal{D}_{\text{cand}}| - d_{\max} + 1)/d_{\max}) \rfloor} \\ &\le (|\mathcal{D}_{\text{cand}}| - d_{\max} + 1)/d_{\max} \\ &\le |\mathcal{D}_{\text{cand}}|/d_{\max} \le d_{\text{int}} m/d_{\max} \\ &\le m\end{aligned}$$

Proof of Theorem V.1. Write $p_{\mathcal{D}_w}$ for the projection of p, and let $\bar{u}$ contain the coefficients of u on $\mathcal{D}_w$. Coverage implies that every omitted coefficient is zero. Thus, $\bar{u} \ge 0$, $\|\bar{u}\|_1 = 1$, and $u \cdot p = \bar{u} \cdot p_{\mathcal{D}_w}$ for every $p \in X$. Consequently,

$$\begin{aligned}\operatorname{rr}_X(S_i, u) &= 1 - \frac{\max_{p \in S_i} \bar{u} \cdot p_{\mathcal{D}_w}}{\max_{p \in X} \bar{u} \cdot p_{\mathcal{D}_w}} \\ &= \operatorname{rr}_{X_{\mathcal{D}_w}}\big((S_i)_{\mathcal{D}_w}, \bar{u}\big) \\ &\le \varepsilon.\end{aligned}$$

The last inequality follows from the assumed projected regret bound. □

d) Trimming retention: Fix u, and condition on the history before postprocessing. For an oversized union of size M, the preceding union contains at most $K-1$ tuples and the last run adds at most k, so $M \le K+k-1$. Uniform trimming retains a specified maximizer of $u \cdot p$ in U with probability $K/M \ge \eta$, where $\eta = K/(K+k-1)$; filling or retaining an undersized or exact-size union preserves every tuple. Thus, writing $G = \{\operatorname{rr}_X(U,u) \le \varepsilon\}$ and $\mathcal{F}$ for this history,

$$\begin{aligned}\Pr\big(\operatorname{rr}_X(S,u) \le \varepsilon\big) &\ge \mathbb{E}[\mathbf{1}_G \Pr(z \in S \mid \mathcal{F})] \\ &\ge \eta \Pr(G),\end{aligned}$$

where $z \in \arg\max_{p \in U} u \cdot p$ is chosen from the history using a fixed rule. Retaining z preserves the union's regret bound.

Proof of Lemma V.2. There are $\binom{D}{w}$ equally likely samples. A sample containing $\mathcal{H}$ is completed by choosing $w-h$ attributes from the remaining $D-h$ attributes. Hence,

$$\begin{aligned}P_{\text{cover}} &= \frac{\binom{D-h}{w-h}}{\binom{D}{w}} \\ &= \frac{\binom{w}{h}}{\binom{D}{h}} \\ &= \prod_{j=0}^{h-1} \frac{w-j}{D-j} \\ &\ge \left(\frac{w-h+1}{D}\right)^h.\end{aligned}$$

The last inequality follows from $w - j \ge w - h + 1$ and $D - j \le D$ for every $0 \le j < h$. □

e) Fixed-round coverage: Each sample fails to cover the key attributes with probability $1 - P_{\text{cover}}$. Independence therefore gives $\Pr(C_t^c) = (1 - P_{\text{cover}})^t$, proving the coverage formula. Solving $(1 - P_{\text{cover}})^t \le 1 - \gamma$ gives $t \ge \ln(1-\gamma)/\ln(1 - P_{\text{cover}})$. Taking the ceiling yields the minimum integer t_γ.

APPENDIX C
CAPABILITY QUALIFICATIONS

a) Comparison criteria: In Table I, practical high-dimensional support concerns practical computation and interaction costs. A half-check denotes substantial cost or a required implementation repair; a cross denotes execution failure in the tested setting. Execution assessments concern the tested HD-PI and VERIFY-POINT implementations; RH and VERIFY-SPACE remain untested.

b) Exact-favorite guarantee: Exact-favorite guarantees assume sufficient feedback: HD-PI/RH use target rank one for favorite retrieval [31], VERIFY-POINT/VERIFY-SPACE provide probabilistic guarantees under their error model [32], and FHDR assumes sparse preferences.

c) Implicit key-attribute discovery: Key-attribute discovery excludes learning weights over an upfront user-selected attribute set.

d) Output size control: Output size control means a user-specified output budget independent of the original dimensionality. Published set-output extensions are included; fixed singleton output is excluded. SPHERE assumes $k \ge d$ [17]. The original UTILITYAPPROX algorithm returns a singleton and does not specify arbitrary-size final output [2]. The HD-PI/RH entries include the published ALLTOPK and SOME-TOPK extensions [31]; the VERIFY-POINT size-control entry includes the published set-output extensions [32]. FHDR's output size control applies before interactive Phase 3 through ATTRIBUTESUBSET; during Phase 3, it returns one recommendation. BREAKPOINT returns an indistinguishability candidate set whose cardinality is not prescribed by the user [33].

e) Output at any interaction budget: This criterion requires a specified recommendation procedure at every interaction budget, including zero feedback. Recommendation quality depends on the available information. Early stopping alone does not establish zero-feedback support.

f) Existing execution and interaction limitations: SPHERE exceeds one hour on the synthetic dataset with $d = 100$ and $n = 10^5$ and requires output size $k \ge d$. UTILITYAPPROX requires approximately 1,100 and 1,149 questions on this synthetic dataset and NBA, respectively, making interaction impractical. Reproduced UH-RANDOM/UH-SIMPLEX and HD-PI encounter numerical failures during convex-hull computation on the targeted high-dimensional inputs.

g) Additional reproduction setting: We tested the released BREAKPOINT and VERIFY-POINT implementations at commits `adf0740` and `22016b7`, respectively, on an Apple M2 Pro with 32 GiB RAM, seed 42, and a one-hour deadline per main attempt. Inputs retained all original dimensions. Times include loading and native preprocessing. These are

individual execution diagnostics; concurrent independent runs preclude interpreting the times as comparative speedups.

h) Breakpoint: With $s = 2$, indistinguishability threshold 0.05, and $q = 5d$, BREAKPOINT completes on NBA ($4{,}790 \times 104$) in 7.27 s using 520 rounds and returns 532 tuples. With 30 or zero rounds, it returns $3{,}813$ tuples. On the $100{,}000 \times 100$ synthetic input, a sanitizer confirms overflow in the published 32-bit slope counter: a tested count is $2{,}493{,}946{,}605 > 2^{31} - 1$. A separate copy with 64-bit counts and corresponding sentinels, retaining the algorithmic procedures, completes in 747.04 s using 500 rounds and returns $18{,}794$ tuples. The original code completes in $1{,}039.12$ s but contains the overflow and returns all $100{,}000$ tuples. On the $10{,}000 \times 100$ input, both versions return all $10{,}000$ tuples. The repair, runtime, interaction burden, and large outputs motivate the half-check. Its additive indistinguishability-error diagnostic is distinct from regret ratio.

i) Verify-Point: Using the supplied driver's error rate 0.05 and checking count 3, VERIFY-POINT returns no recommendation on either principal input. NBA logs Qhull allocation error `QH6235` after generating approximately 7.7 million facets. The NBA and $100{,}000 \times 100$ synthetic attempts are stopped during hull construction after 697.90 s and 794.68 s, with measured memory footprints of 17.7 and 15.4 GiB, respectively. These are termination times, not completed runs or one-hour timeouts. A supplied 100×100 input triggers feasible-point parsing error `QH6072` followed by a segmentation fault with both error-free and 5% erroneous responses. Library paths were adapted and timing drivers added; the tested algorithm and geometry routines were unchanged. These results characterize the released implementations and tested settings.

APPENDIX D
ADDITIONAL EXPERIMENTAL RESULTS ON REAL DATASETS

We present here the additional experimental results of four real datasets, namely House, Car, Energy and Activity. The default values of q are 5, 9, 22, 8 respectively. The results can be found in Figures 13, 14, 15 and 16.

APPENDIX E
SUPPLEMENTARY MATERIAL FOR EXPERIMENTAL SETTING

We evaluate internal parameter choices in terms of regret ratio and execution time.

To assess m, we measure the number of questions required to identify the single tuple with the highest utility. As observed in Figure 18, the number of questions required to identify the optimal tuple decreases as m increases, since each question yields more information. However, a larger m also makes each question more cognitively demanding for the user. We must balance these factors, as both the number of questions and the number of attributes per question contribute to overall cognitive load. Cognitive science literature indicates that humans can only comfortably process a limited number of items at once (e.g., 7 as suggested by [52]), and recent working memory studies find capacity constraints on the order of $4 \sim 7$ [53]. In parallel, marketing research guidelines for conjoint analysis recommend using roughly $5 \sim 7$ attributes at most in each task to avoid overwhelming respondents [54], also supported by some consumer research [55], [56]. Therefore, we use m at 7 in our approach, as this upper limit balances query efficiency in our algorithm with user cognitive limits, aligning with the above guidelines.

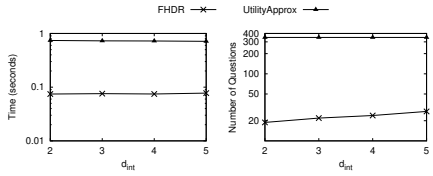
TABLE V: User study results

P	FHDR			UH-Random			Mech. Rate		Pref.
	A	S	T	D	A	T	F	S	
1	30	20	147	3	4	57	6	4	F
2	13	8	119	6	10	86	10	10	U
3	0	0	0	2	4	59	4	7	B
4	0	0	0	3	7	92	5	7	B
5	2	1	15	3	4	72	6	6	U
6	3	1	18	3	3	49	8	8	F
7	16	9	74	4	8	124	9	7	F
8	0	0	0	2	5	57	6	5	U
9	30	12	198	4	7	98	7	7	F
10	27	15	108	3	8	99	6	5	B
11	13	6	98	4	8	102	7	6	F
12	12	6	67	6	6	67	7	6	F
13	0	0	0	3	6	91	7	3	U
14	30	11	189	5	8	132	8	9	U
15	9	6	90	3	6	64	8	8	F
16	13	9	82	7	7	88	6	3	F
17	9	6	52	2	5	77	7	6	F
18	15	5	112	3	7	85	5	6	B
19	10	5	45	4	6	99	5	8	U
20	5	3	21	5	11	76	7	7	F
21	33	21	182	3	8	117	9	4	F
22	1	1	9	5	7	73	9	5	F
23	31	21	110	3	9	112	10	5	F
24	28	7	217	5	9	112	9	9	F
25	32	15	174	2	7	77	4	8	U
26	5	3	76	5	12	114	8	7	F
27	28	9	176	3	9	119	7	5	F
28	31	8	199	7	12	150	8	6	F
29	0	0	0	4	7	86	8	8	U
30	12	6	81	4	8	81	7	6	F
AVG	14.60	7.13	88.63	3.87	7.27	90.50	7.10	6.37	F:18 U:8 B:4

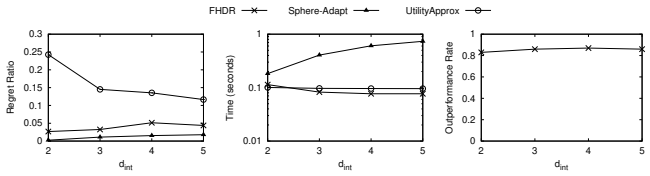
For w, we restrict the number of questions so that the algorithm can only finish Phase 1, to evaluate performance in ATTRIBUTESUBSET. We plot the execution time on the Energy dataset, and the regret ratio on three datasets (from left to right): Car, NBA, Energy. Results are presented in Figure 17. As shown in the first plot, the execution time increases sharply once w exceeds 6. Regarding the regret ratio, a larger w raises the likelihood of including the *key* attributes within each sampled subset, while simultaneously reducing the accuracy upon successfully including all *key* attributes. These two effects counterbalance each other, leading to a more intricate, non-monotonic curve rather than a steady trajectory. In particular, there is a local valley at $w = 6$, followed by a peak around $w = 7 \sim 8$. Based on the overall performance, we select $w = 6$ that achieves both efficiency and accuracy.

APPENDIX F
ADDITIONAL STATISTICS OF USER STUDY

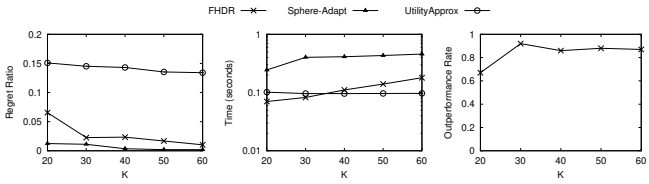
We gathered 30 participants for the user study, mostly university students. Detailed statistics from the user study are



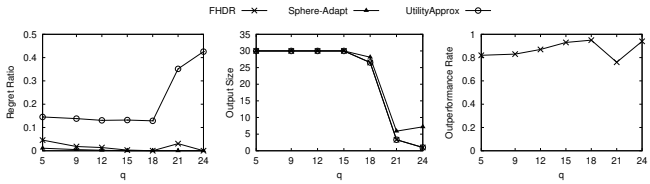
(a) P1: Vary d_{int}



(b) P2: Vary d_{int}

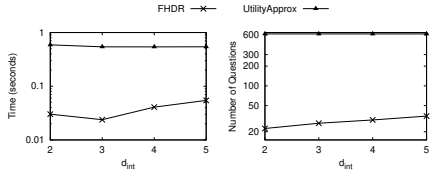


(c) P2: Vary K

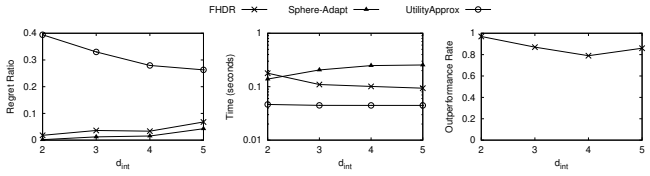


(d) P2: Vary q

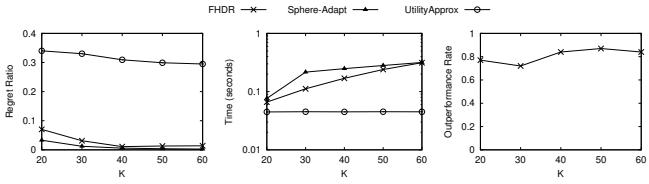
Fig. 13: Results on *House* dataset



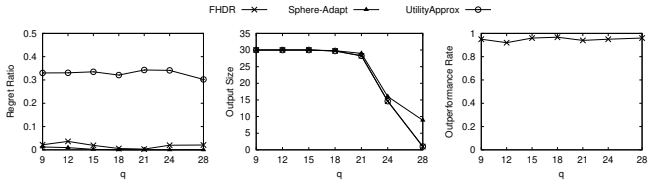
(a) P1: Vary d_{int}



(b) P2: Vary d_{int}

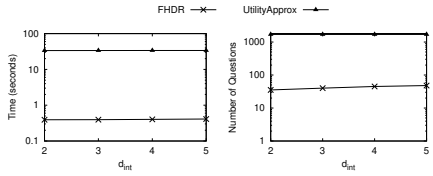


(c) P2: Vary K

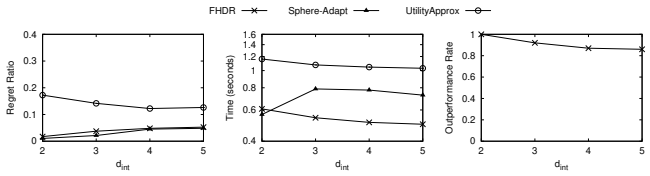


(d) P2: Vary q

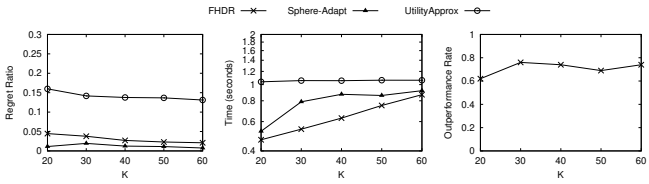
Fig. 14: Results on *Car* dataset



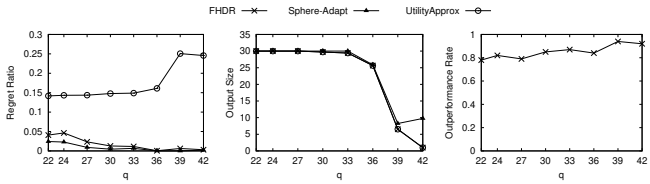
(a) P1: Vary d_{int}



(b) P2: Vary d_{int}

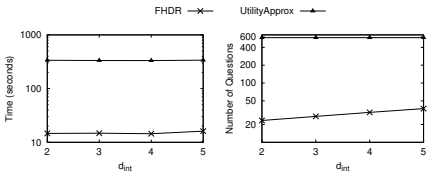


(c) P2: Vary K

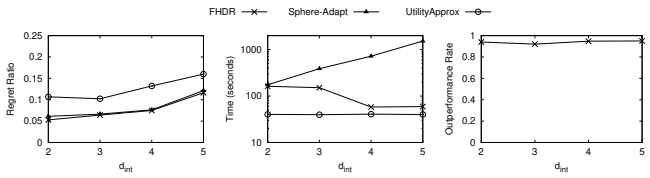


(d) P2: Vary q

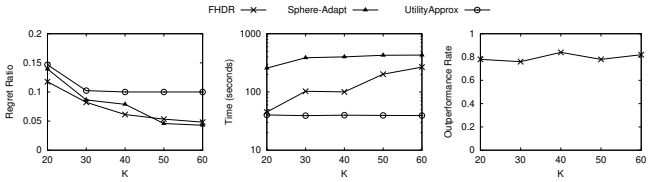
Fig. 15: Results on *Energy* dataset



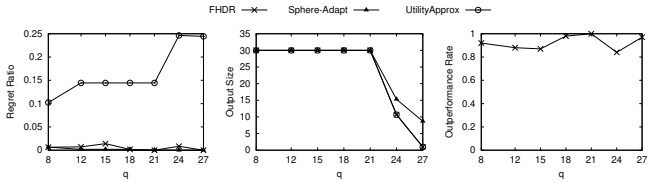
(a) P1: Vary d_{int}



(b) P2: Vary d_{int}

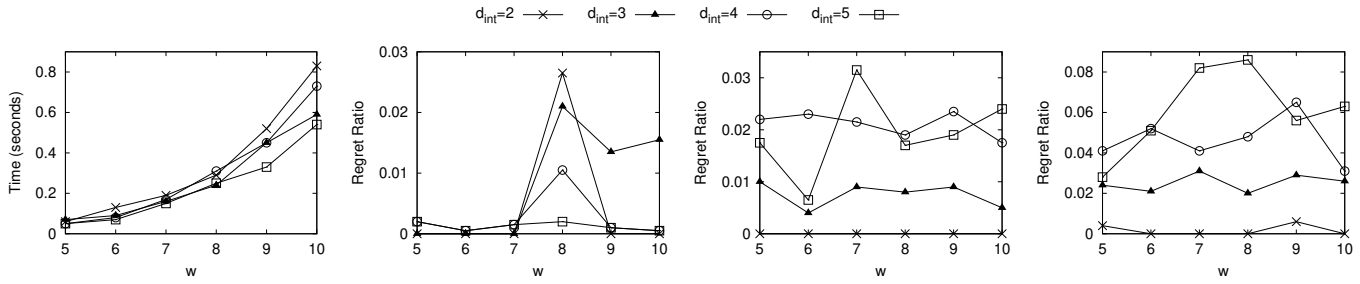


(c) P2: Vary K



(d) P2: Vary q

Fig. 16: Results on *Activity* dataset



(Energy) (Car) (NBA) (Energy)

Fig. 17: Vary w

Dataset	d	$\lvert X\rvert$
House	33	15,171
Car	57	5,500
NBA	104	4,790
Energy	149	36,043
Activity	54	2,872,533

TABLE IV: Real Datasets

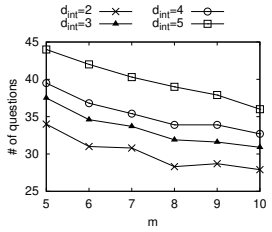


Fig. 18: Vary m

presented in Table V. The two main parts in the table, FHDR and UH-RANDOM, records detailed statistics of a specific user using the algorithm. Due to limited space, we denote 'P' as participant, 'A' as the number of questions answered (including those being skipped), 'S' as the number of skipped questions, 'T' as the recorded time using the algorithm in seconds, including upfront attribute selection for UH-RANDOM, and 'D' as the number of attributes selected for the UH-RANDOM algorithm, as required. For FHDR sessions with no questions, $T = 0$ denotes zero interaction time. Both algorithms contain only three columns: FHDR contains (A,S,T) as we do not know the number of attributes that the user considers implicitly while using the algorithm, and UH-RANDOM contains (D,A,T) since skipping is not allowed. The remaining columns report the ratings of the two interaction mechanisms and the preference over recommendations returned by FHDR and UH-RANDOM.

Upfront attribute-selection time for UH-RANDOM was explicitly measured at approximately 50 s and is included in its mean total time of 90.50 s. FHDR's mean recorded time is 88.63 s. While the total times are similar, FHDR avoids upfront attribute selection and allows users to skip questions or stop answering at any time. Participants 3, 4, 8, 13, and 29 requested recommendations without answering questions and received ATTRIBUTESUBSET outputs. Their $T = 0$ entries denote zero interaction time. Other participants skipped questions during interaction, illustrating the use of FHDR's flexible feedback options. Finally focusing on the preference over the final results, most participants favor the recommendation of our algorithm, which is either more accurately reflecting their preference, or could cover some preferred options successfully.